

Chapter 8 8086 Interrupts

Dr. Prof. Ni Li

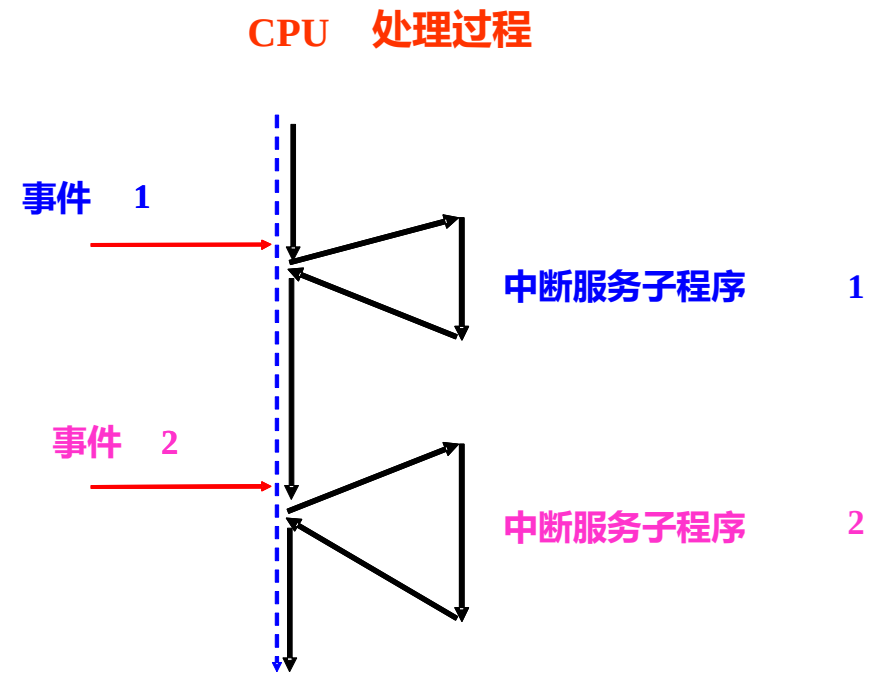
扫码答题



基本概念介绍

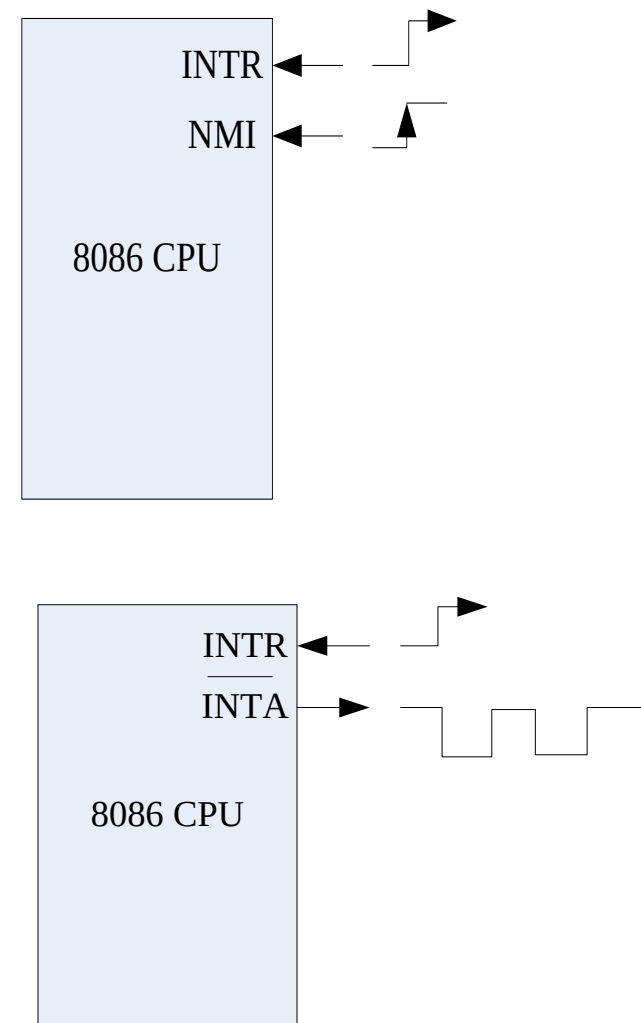
基本概念介绍

- Interrupt 中断
 - CPU 由内部或者外部事件打断
 - CPU 停止执行当前程序，转而执行为该中断“服务”的一段特殊程序
 - 一旦处理完成，CPU返回被打断的地方继续执行



基本概念介绍

- Interrupt request signal 中断请求信号
 - 可屏蔽中断请求 INTR
 - 不可屏蔽中断请求 NMI
- Interrupt response 中断响应
 - NMI->响应中断请求
 - INTR-> 根据IF响应中断请求
- Interrupt sources 中断源
 - 引起中断的内部或外部原因
 - 硬件（外部中断请求）或者软件



基本概念介绍

- Interrupt service routine/procedure 中断服务程序
 - 提前准备好，当中断发生时执行
- Interrupt service routine address 中断服务程序地址
 - 中断服务子程序地址: 2-byte 偏移地址+2-byte 段地址
 - Interrupt vector 中断向量

基本概念介绍

- Breakpoint address 断点地址
 - CPU被打断时**下一条指令地址，也是**中断执行后的返回地址
 - 2-byte 偏移地址， 2-byte 段地址

断点概念：

1000:150H →

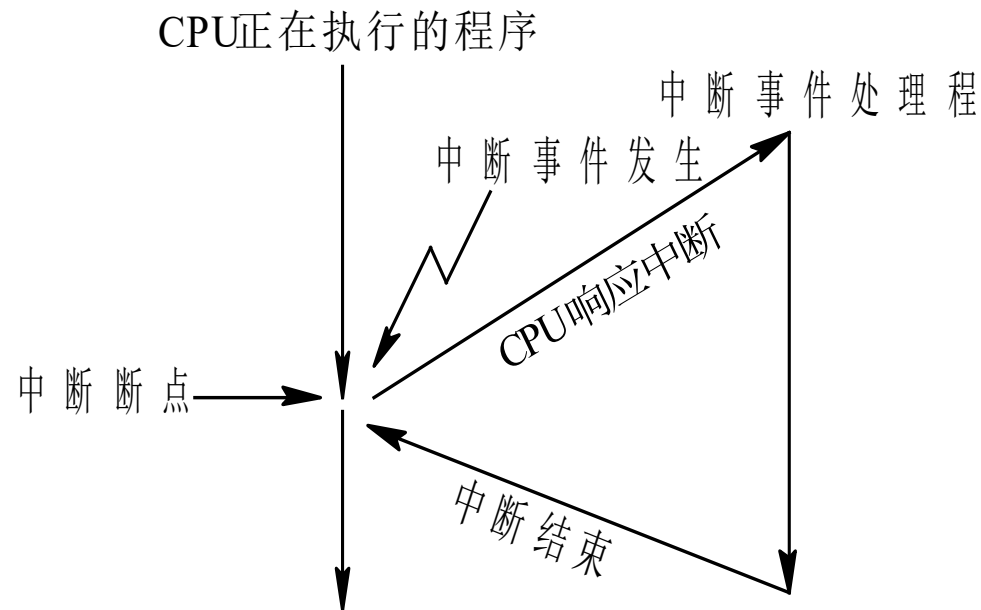
```

\ \
\ \
\ \
\ \
MOV  AX, 0
ADD  AX, DX
MOV  [DI], AX
\ \
\ \
\ \
\ \
PUSH AX
\ \
\ \
IRET
```

CPU在执行此指令时，
某中断源发申请中断；
CPU在执行完该指令
后，
转去执行中断子程
地址1000:150H为断点

基本概念介绍

- Return from the interrupt 中断返回
 - 当完成中断服务子程序执行后，CPU返回之前被打断的地方（断点）继续执行程序

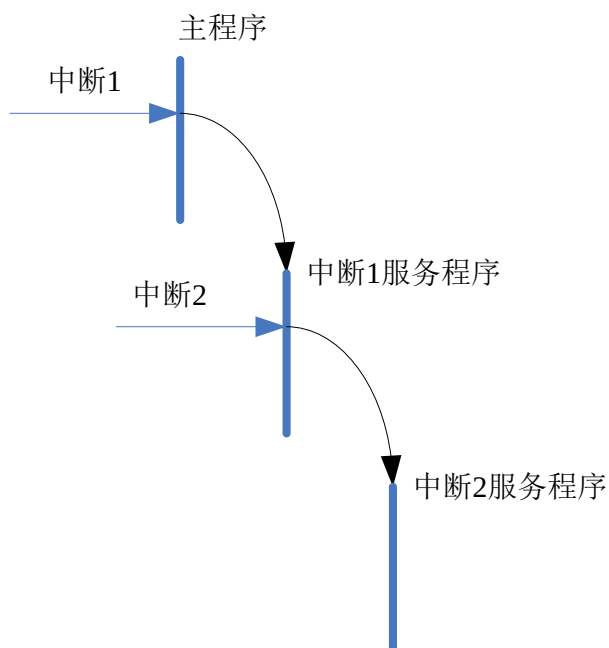


基本概念介绍

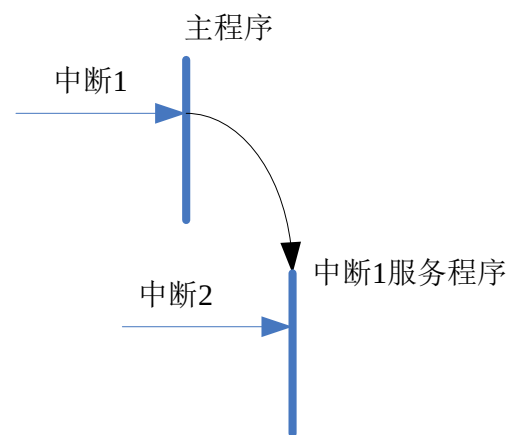
- Enable interrupt/Unable interrupt 开/关中断
 - IF=1, 允许**可屏蔽中断**
 - IF=0, 不允许可屏蔽中断
 - STI(IF=1), CLI(IF=0)
- Interrupt source identification 中断源识别
 - 256 个中断源, 通过中断类型号识别
 - 中断类型号: 8-bit 二进制数, 00H-FFH
- Interrupt priorities 中断优先级
 - 定义每个中断源的优先级等级
 - 多个中断发生时, CPU根据中断优先级来进行响应

基本概念介绍

- Interrupt nesting 中断嵌套
 - 当执行中断程序时，有更高优先级中断产生
 - 转而执行具有更高优先级的中断服务子程序



中断2优先级 > 中断1优先级



中断2优先级 < 中断1优先级

基本概念介绍

- 中断现场
 - CPU执行中断服务子程序前的状态，包括CPU寄存器及断点地址
- 保护现场
 - 当中断响应时，CPU自动完成对**FR**, 断点地址(**段地址和偏移地址**) 的入栈保存
 - 用户: 将中断服务子程序中用到的寄存器入栈保存, **Example**
- 恢复现场
 - 当中断服务子程序执行完毕，CPU 自动从堆栈中把断点地址(**给IP, CS**) 和**FR**出栈

主程序:

....

MOV **AL**, 12; 产生中断

ADD **AL**, 34

.....

中断服务程序:

MOV **AL**, [BX]

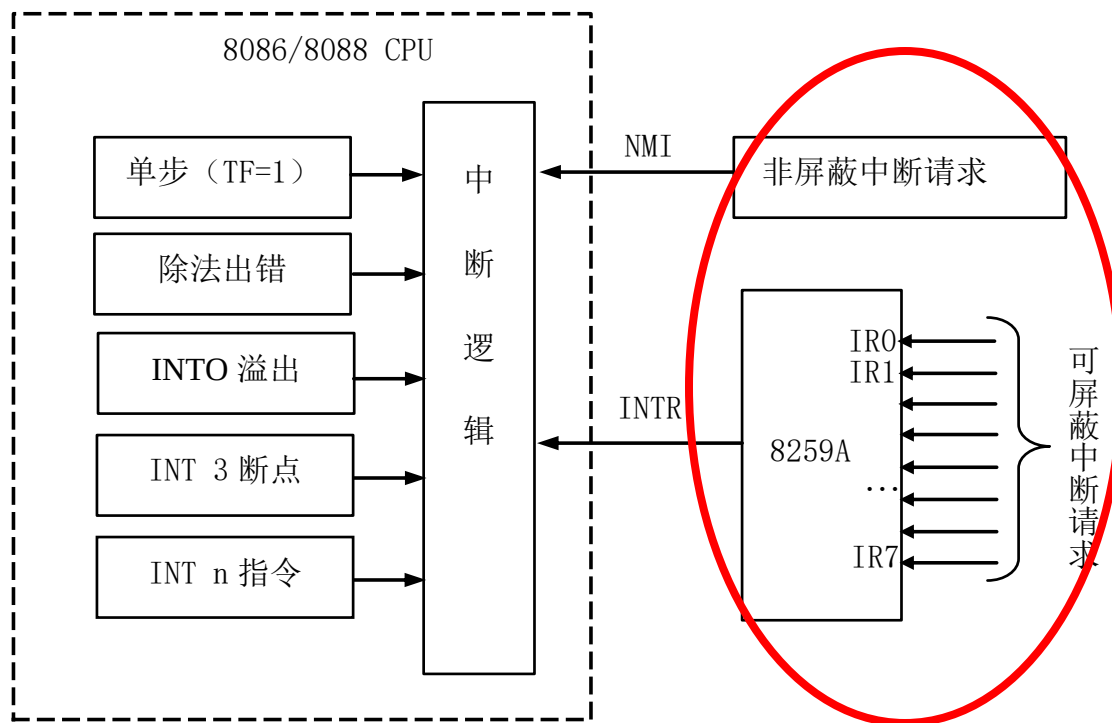
OUT DX, **AL**

.....

8086 中断系统

8086 中断源：256个

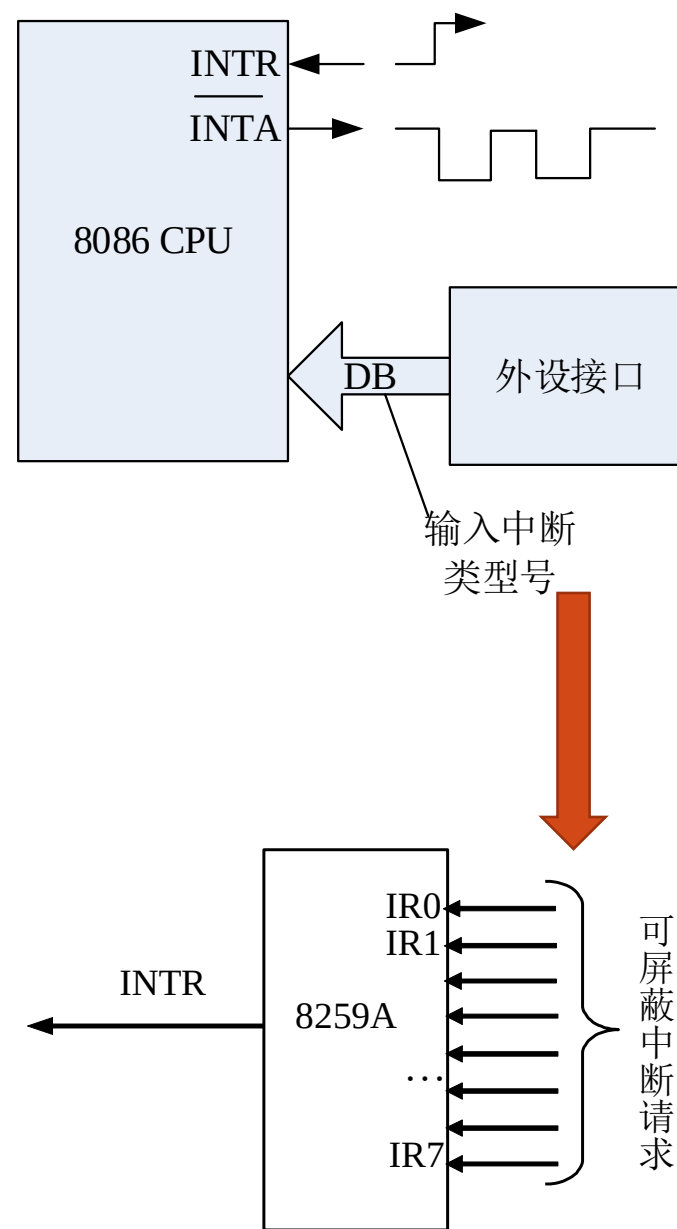
- 外部中断 (硬件中断)
 - 从CPU管脚传入信号
 - INTR (IF)
 - NMI
- 内部中断(软件中断)
 - Debug
 - CPU 计算错误
 - 特殊指令: INT



中断源识别：中断类型号

外部可屏蔽中断

- INTR
 - 2 个低电平: INTA (2 个总线周期)
 - 由外部接口提供中断类型号
- 8259A
 - 可编程中断控制器
 - 管理外设中断请求: 通过中断管理逻辑判断优先级后, 输出中断请求信号INTR给8086
 - 一片8259A能管理8个中断请求, 支持主从级联



NMI

- 不能由IF 屏蔽
- Non-maskable interrupts
 - 硬件错误造成
- 中断类型号： 2

内部中断

- 除0错
 - 中断类型号： 0
- 单步中断
 - 中断类型号： 1, TF=1
- INT 3 断点中断
 - 中断类型号： 3
- INTO 溢出中断
 - 中断类型号： 4, OF=1
- INT N 指令中断
 - 中断类型号： N
 - INT 21H

Example



MOV AX, 1234H

MOV CL, 0

DIV CL

.....

MOV AL, 7FH; 127

ADD AL, 1; 不一定产生中断

INTO; 紧跟溢出中断指令INTO
; 产生类型为4的中断

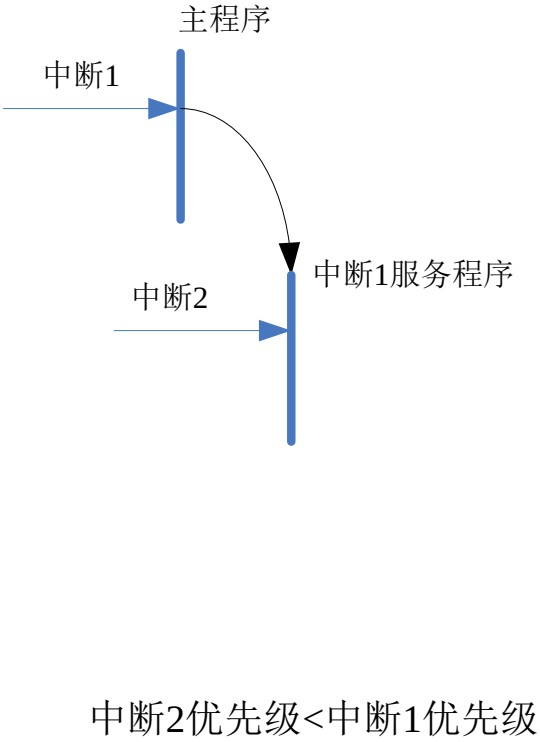
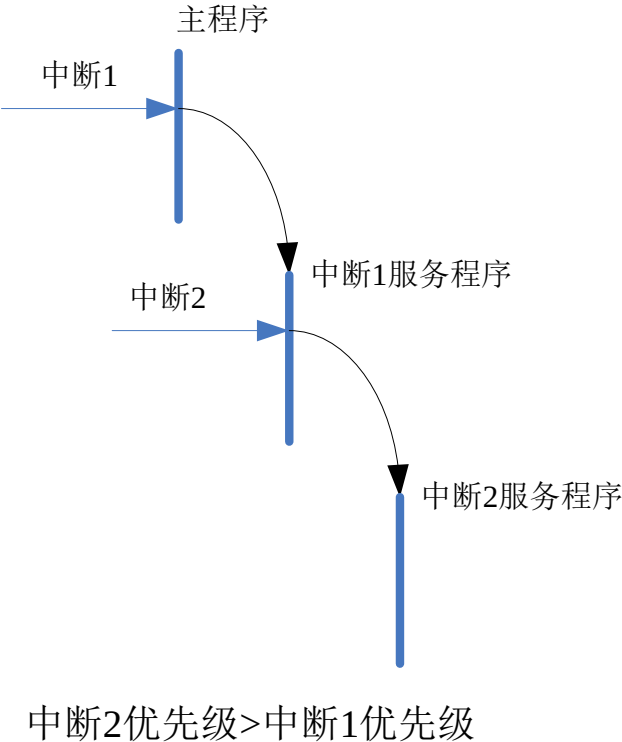
.....

Example



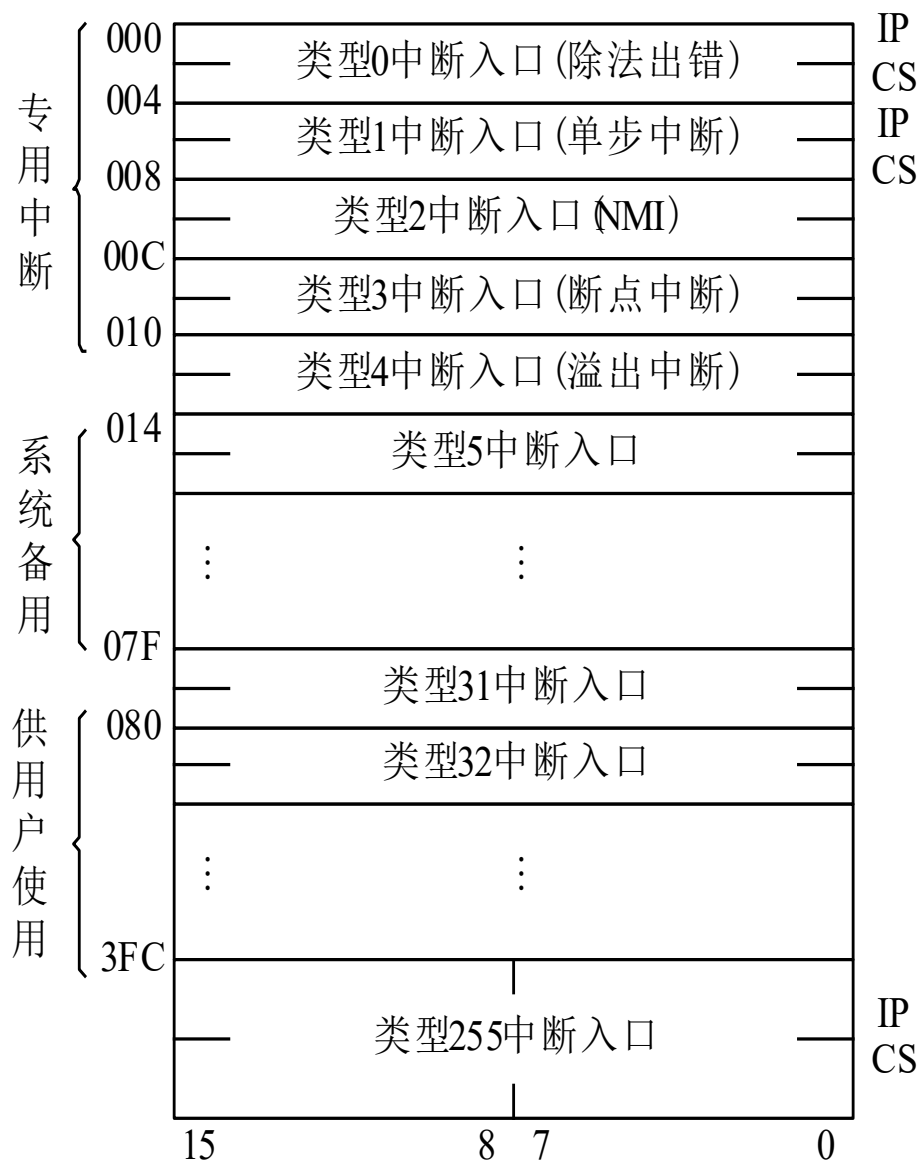
中断优先级

- 中断嵌套？



8086 中断向量表——存储中断向量

Jmp Call 指令：指令中给出跳转地址



- 中断向量（入口地址）
 - 偏移地址(->IP)
 - 段地址(->CS)
- 中断向量表
 - 放256个中断向量:1KB
 - 00000H -- 003FFH
- 中断类型号->中断向量
 - $N \times 4$
 - $4N, 4N+1 \rightarrow IP$
 - $4N+2, 4N+3 \rightarrow CS$

中断向量的存储和管理

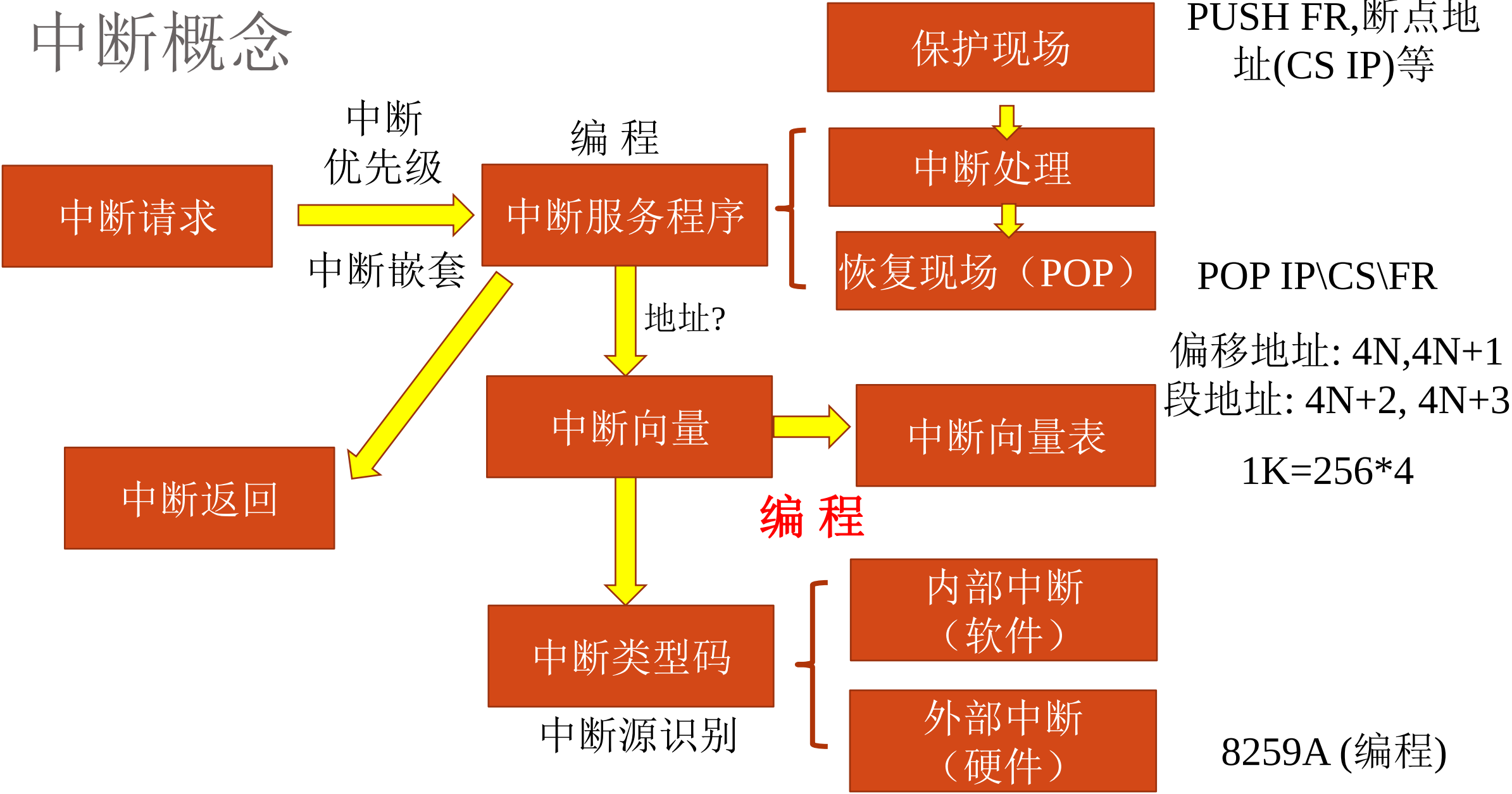
- 起始地址?
 - **examples:** INT 21H
 - 中断类型号: 73H
 - 左移2位



中断类型号

- 除零错, 单步, NMI, INT 3, INTO: 类型号0~4
- INT N: 类型号N
- INTR: 由接口硬件电路提供(e.g. 8259A)

中断概念



8086 应答过程

- 响应INTR—外部可屏蔽中断
 - INTA (2 个总线周期)
 - 取中断类型号N(8259A提供)
 - FR入栈
 - 关中断(Automatically)
 - 保护断点(段地址,偏移地址入栈保存)
 - 获取中断向量
 - 执行中断服务程序(允许中断嵌套可开中断)
 - 中断返回(断点偏移地址->IP, 断点段地址->CS, FR)
- 响应其他中断—已知中断类型号
 - 类似 (除了 $\overline{\text{INTA}}$)
 - 已知中断类型号

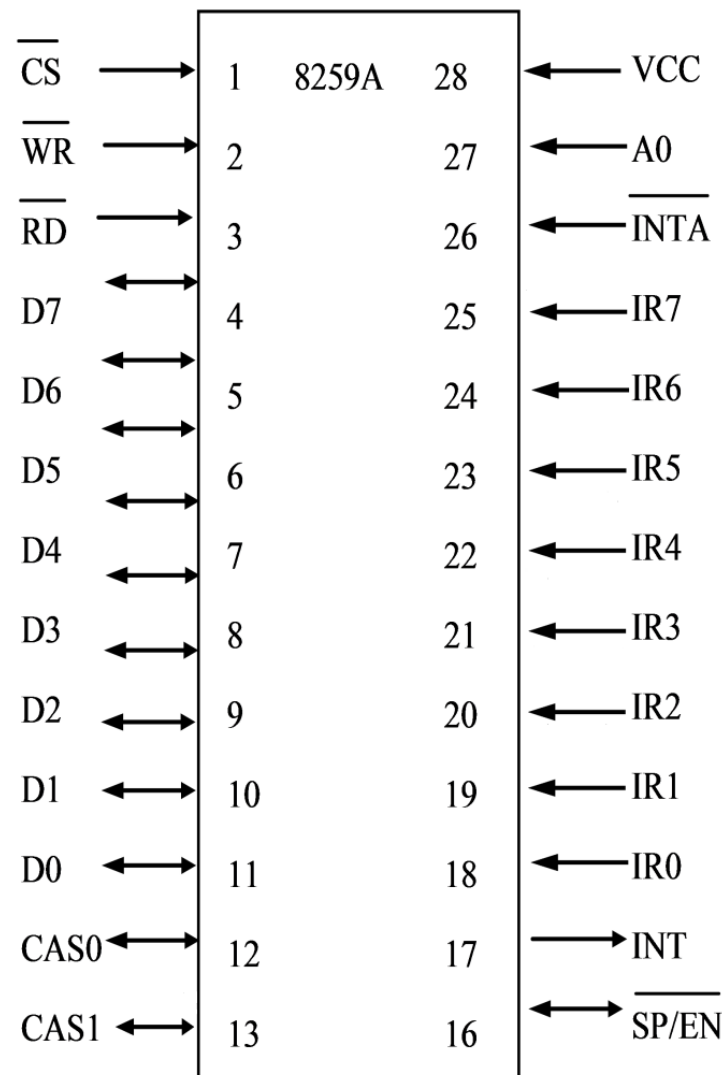
中断控制器8259A

学习芯片

- 工作原理/内部结构
- 管脚定义
- 电路连接 (与CPU 总线)
- 工作方式/时序图
- 编程控制(初始化/ 设置工作方式)

8259A 管脚

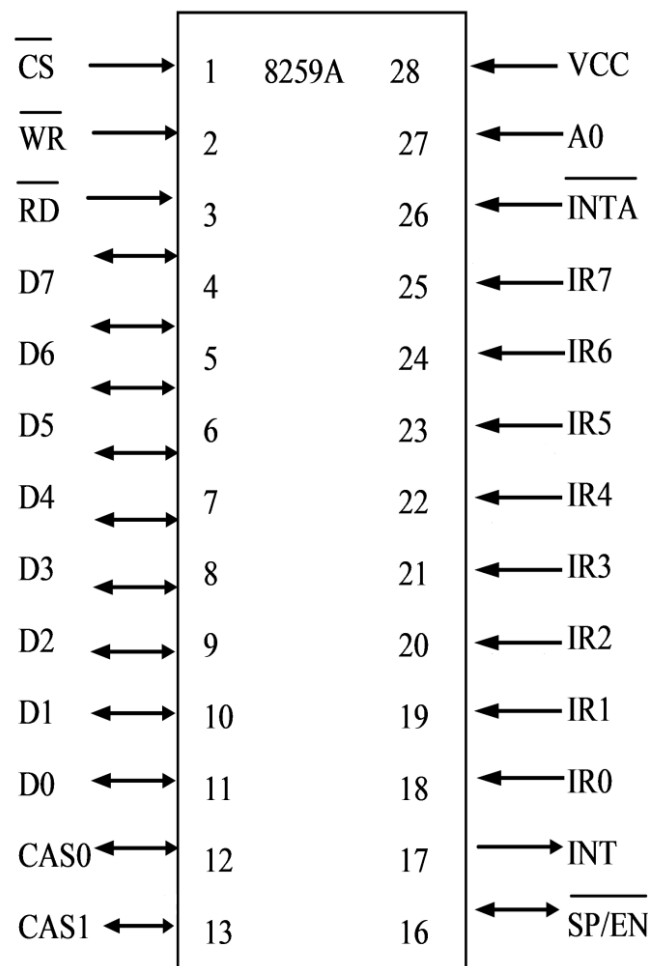
- 28管脚 DIB(Dual In-line Package)
- **D0~D7**: 8位双向数据线; 与CPU传输数据
- **INT**: Interrupt request, output; 从8259A 到CPU 或 从8259A从片到8259A主片
- **$\overline{\text{INTA}}$** : 中断应答, 输入; 来自 CPU
- **IR0~IR7**: 中断请求输入, 从外设或者8259A从片



8259A管脚

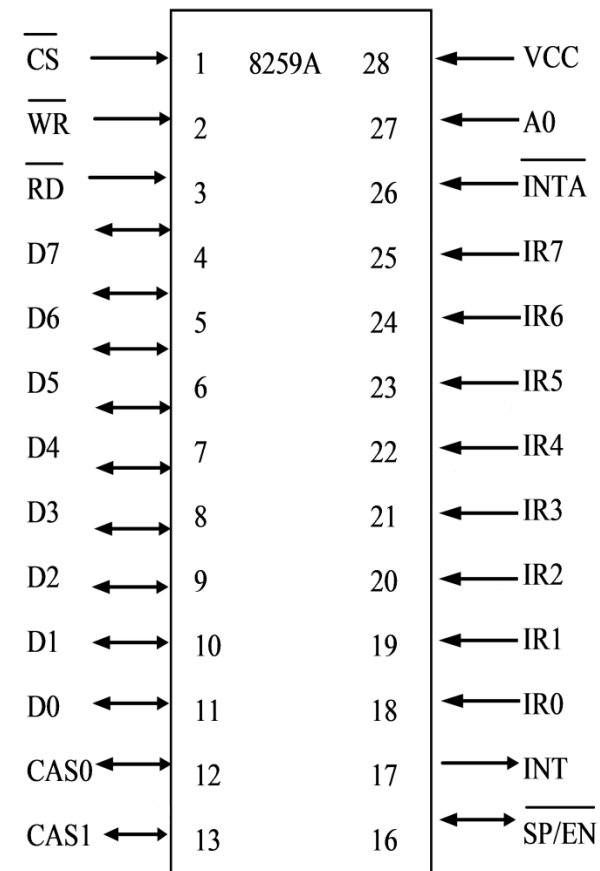
- **CAS0~CAS2**

- 级联信号
- 主片8259A: 输出3-bit (000~111) 识别码选择——8片从片
- 从片8259A: 输入信号, 判断是否被选中 (提前编程设置好识别码)

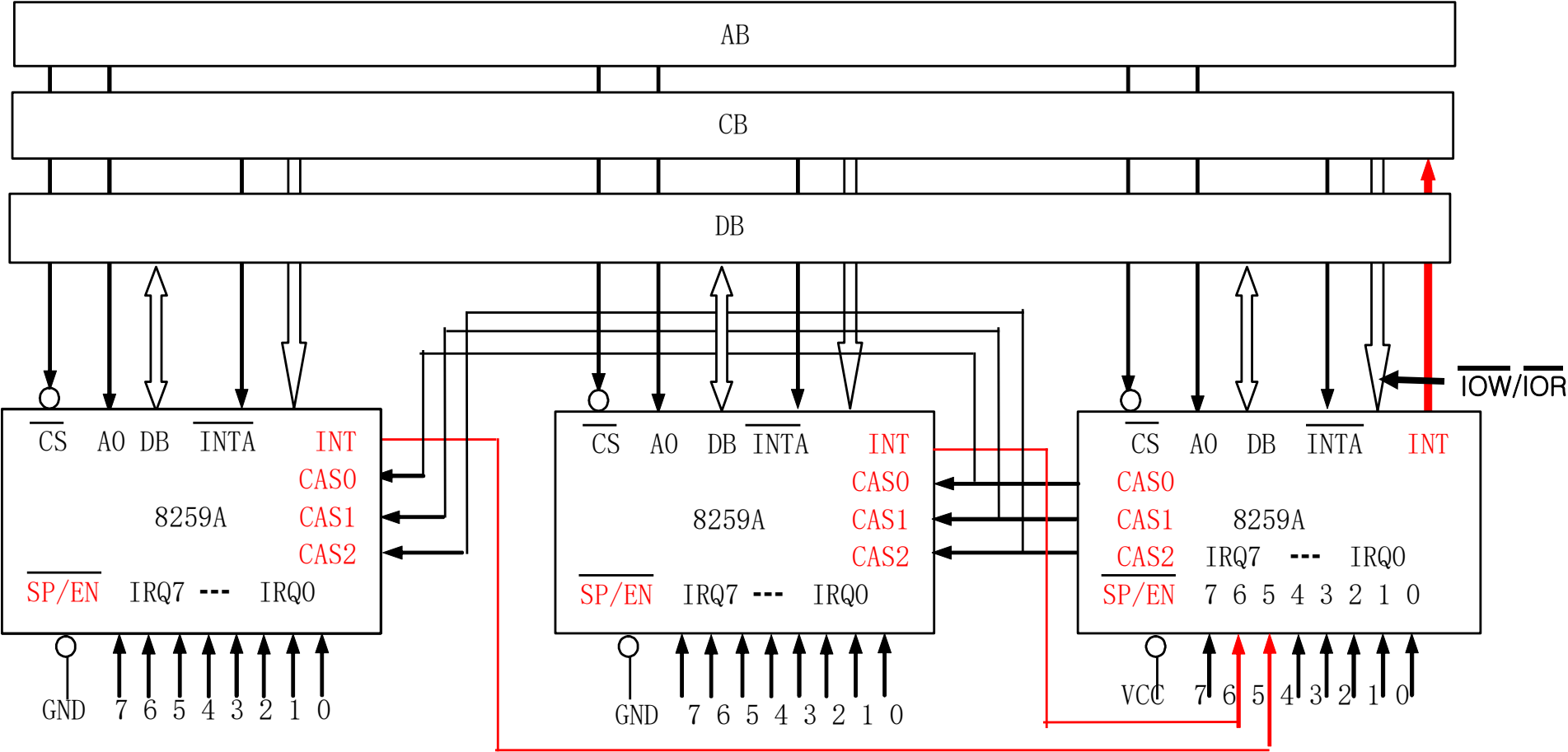


8259A管脚

- $\overline{SP}$ / $\overline{EN}$: slave program/
enable buffer
 - $\overline{SP}=1$, 作主片; $\overline{SP}=0$, 作从片
 - 此时8259A与数据总线相连



级联连接图

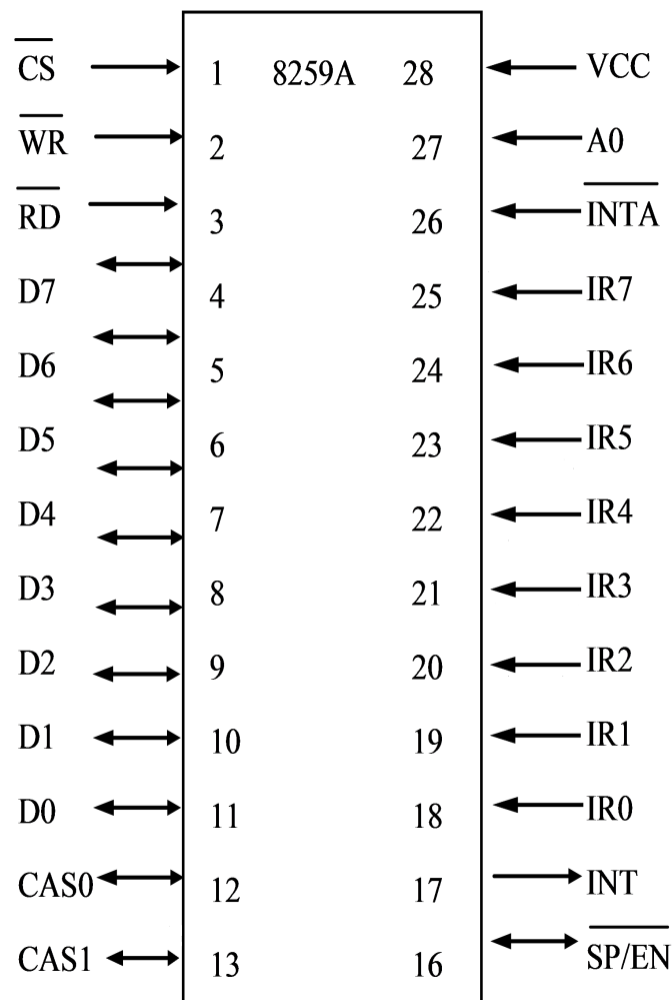


1个主片

2个从片（级联信号？）

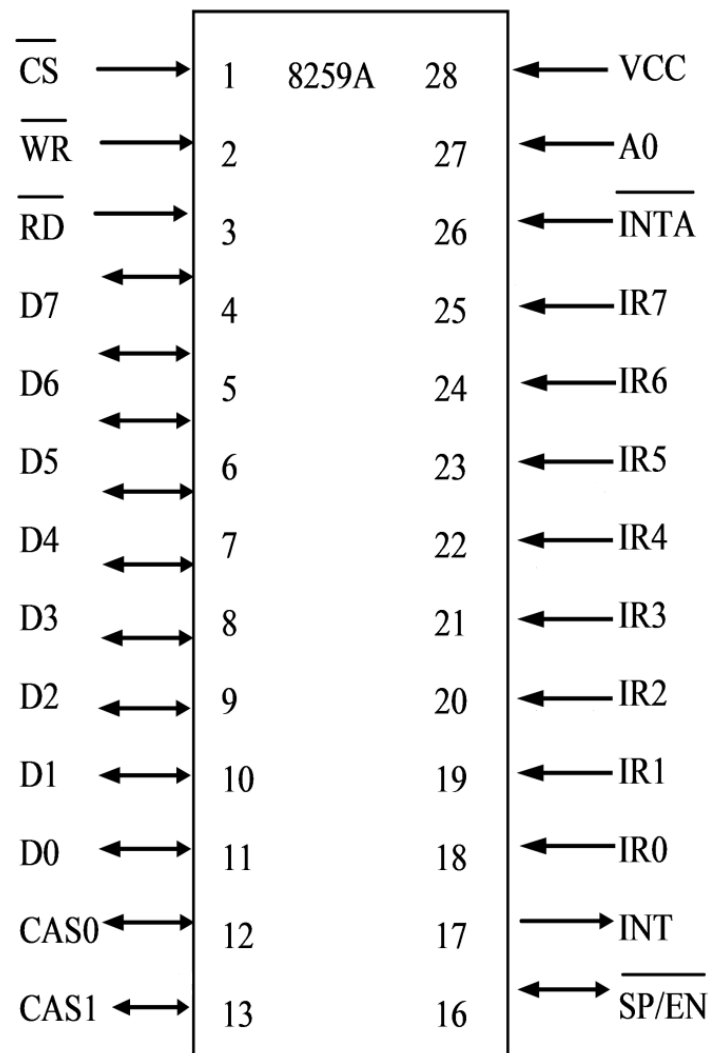
8259A管脚

- A_0 : 内部寄存器选择控制, 地址选择输入
 - 占用端口地址?
 - $A_0=0$, ICW1, OCW2 and OCW3
 - $A_0=1$, ICW2~ICW4 and OCW1
- $\overline{CS}$: 片选信号



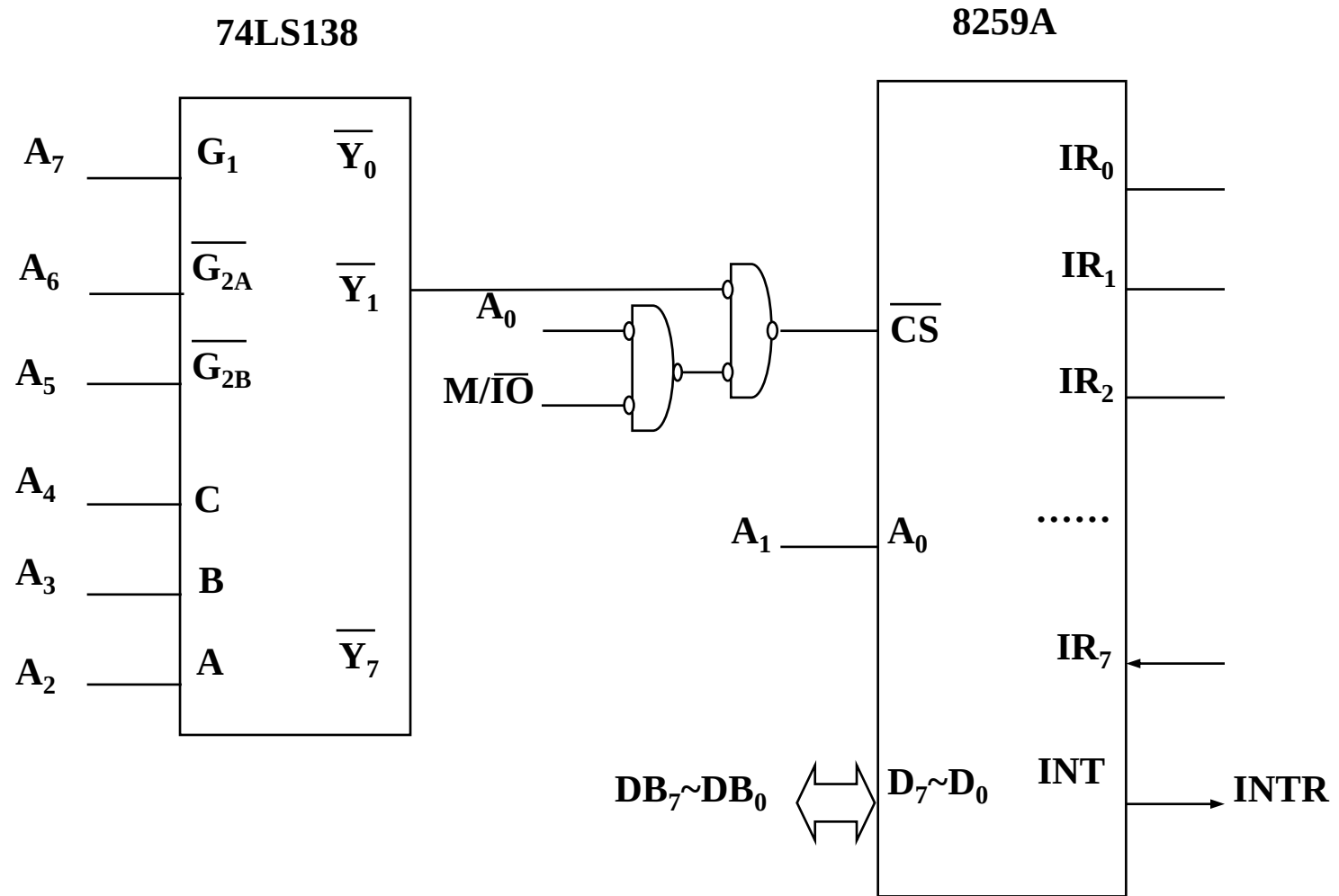
8259A管脚

- $\overline{\text{RD}}$: 输入
 - 来自CPU的 $\overline{\text{IOR}}$
- $\overline{\text{WR}}$: input
 - 来自CPU的 $\overline{\text{IOW}}$



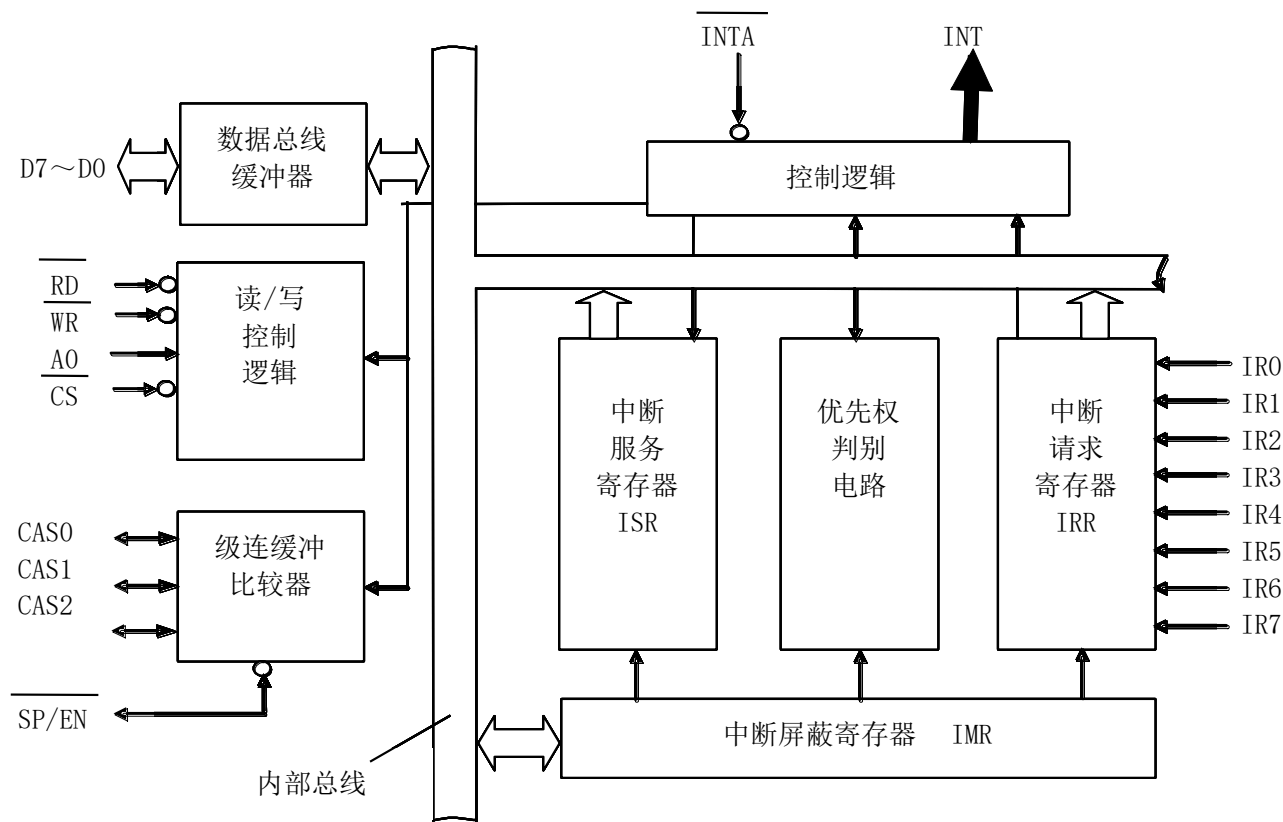
8259A地址生成电路

- 端口地址



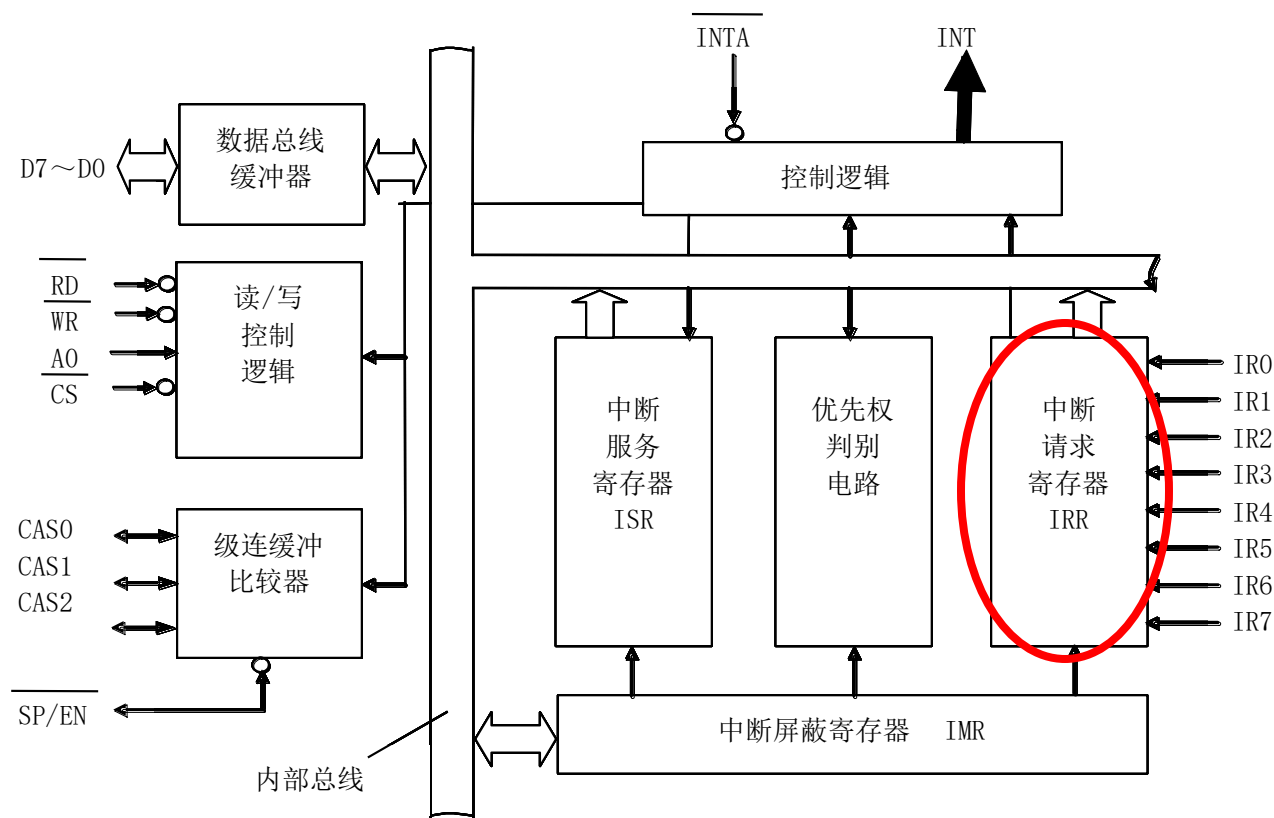
内部结构

- 8 个部件
 - Interrupt request register (IRR)
 - Interrupt service register (ISR)
 - Interrupt mask register (IMR)
 - 优先权判断器
 - 数据总线缓冲器
 - 控制逻辑
 - R/W逻辑
 - 级联缓冲器



内部结构

- IRR:中断请求寄存器
 - 8-bit 寄存器
 - 对应管脚: IR0-IR7
 - 若IRi请求, 则IR Ri=1
 - 若响应IRi中断请求, 对应IRR复位



	IRR7			IRR4			IRR0	
IRR	0	0	0	0	0	0	0	1
	0	0	0	1	0	0	0	0

CPU响应IR0中断请求后IRR0复位

内部结构

- **ISR: 中断服务寄存器**

- **ISR: 8-bit 寄存器**

- 若 $ISR_i = 1$ ，则表示8086正在响应 IR_i 的中断请求，即执行响应的中断服务程序

- 与新进入的中断请求进行优先级比较，从而决定是否进行嵌套

- **中断服务完成，应把ISR对应位复位（清ISR）**

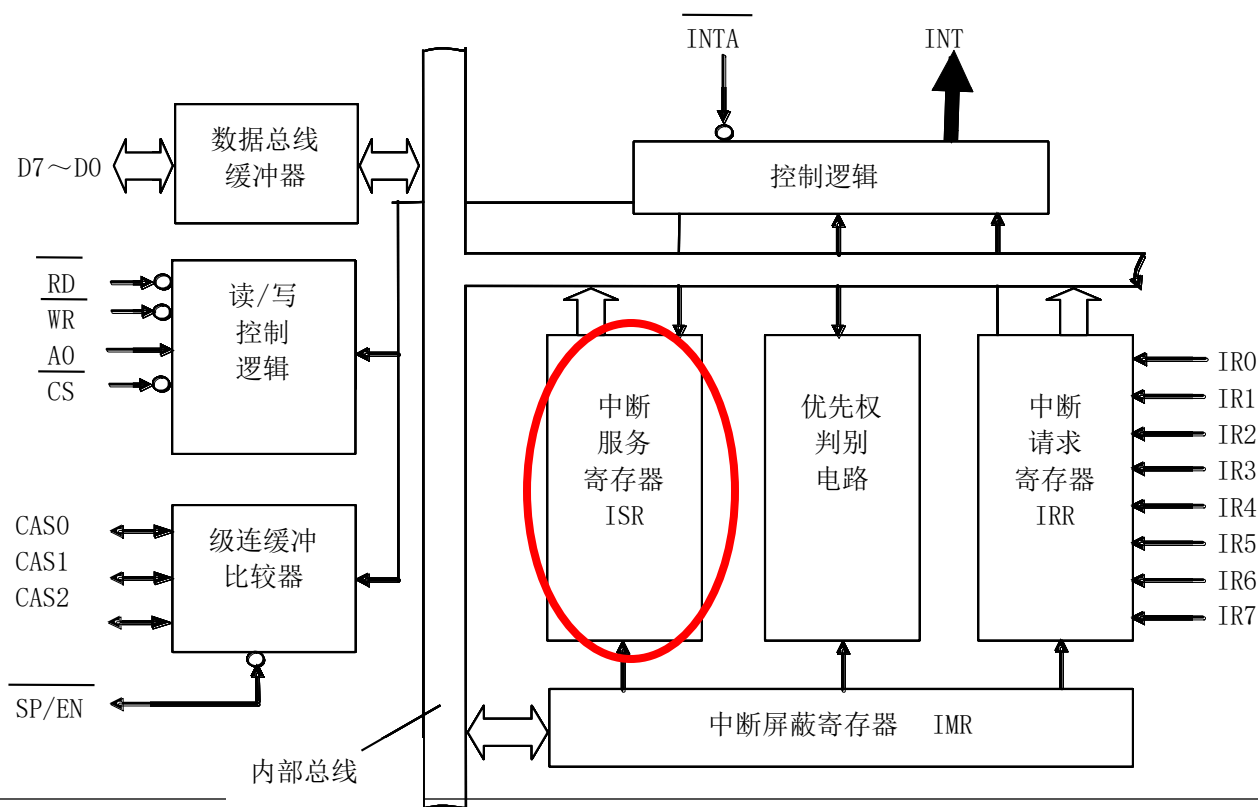
	ISR7				ISR0			
ISR	0	0	0	1	0	0	0	0

	ISR7				ISR0			
IRR	0	0	0	0	0	0	1	0

	ISR7				ISR0			
ISR	0	0	0	1	0	0	1	0

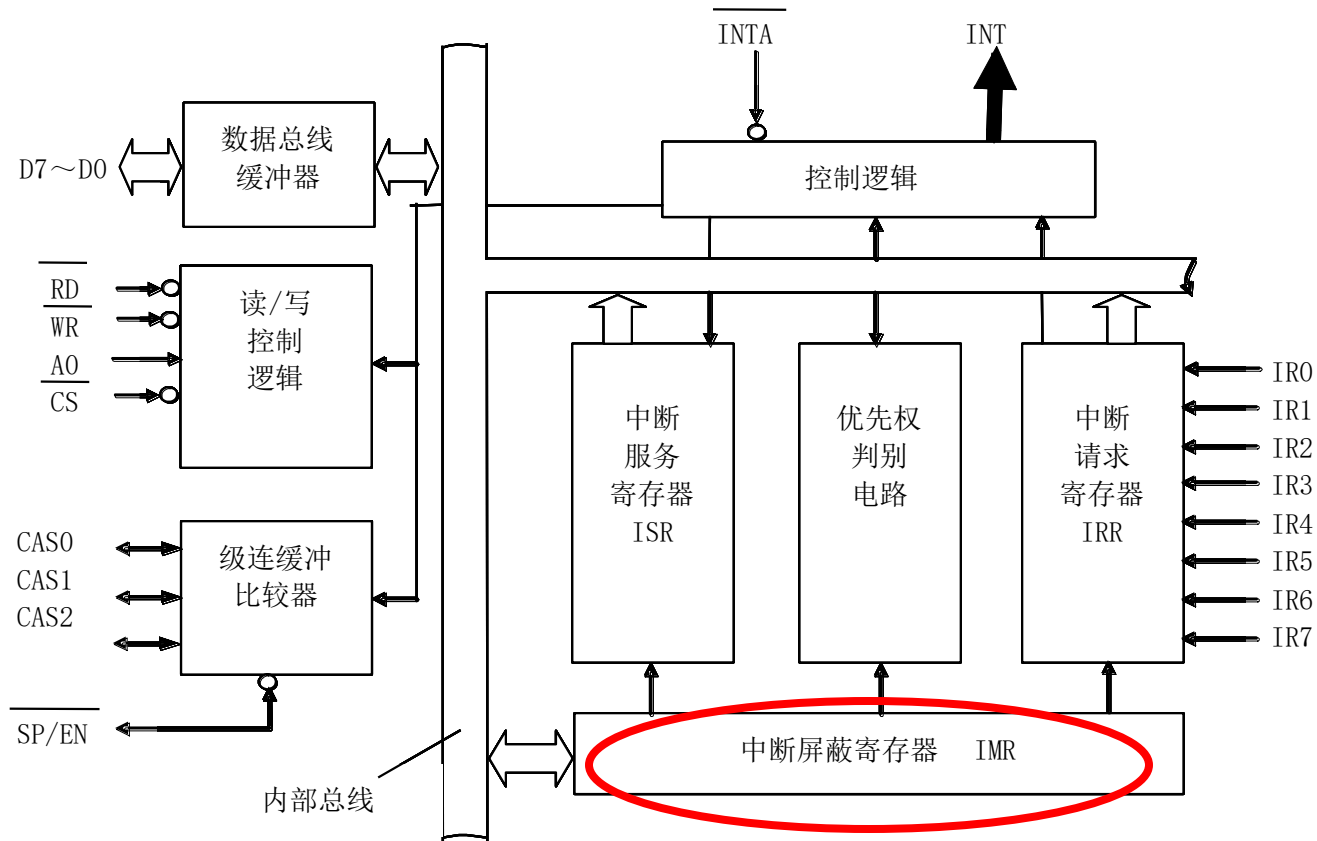
	ISR7				ISR0			
IRR	0	0	0	0	0	0	0	0

进入中断
嵌套



内部结构

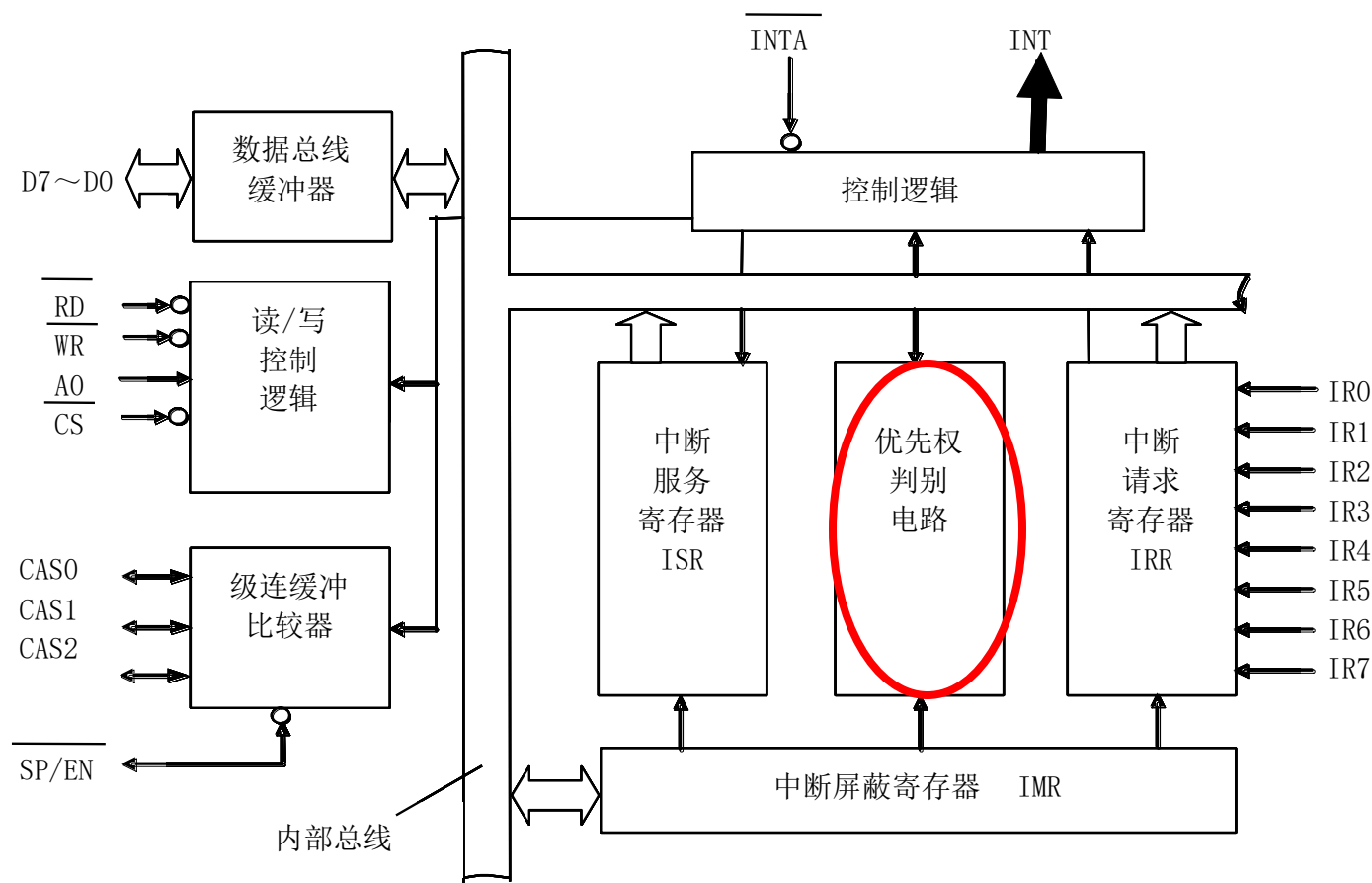
- 中断屏蔽寄存器
- IMR: 8-bit 寄存器
 - 若 $IMR_i = 1$, 则屏蔽 IR_i 的请求, 禁止来自 IR_i 的中断



	IMR7				IMR0			
IMR	0	0	0	0	1	0	1	0

内部结构

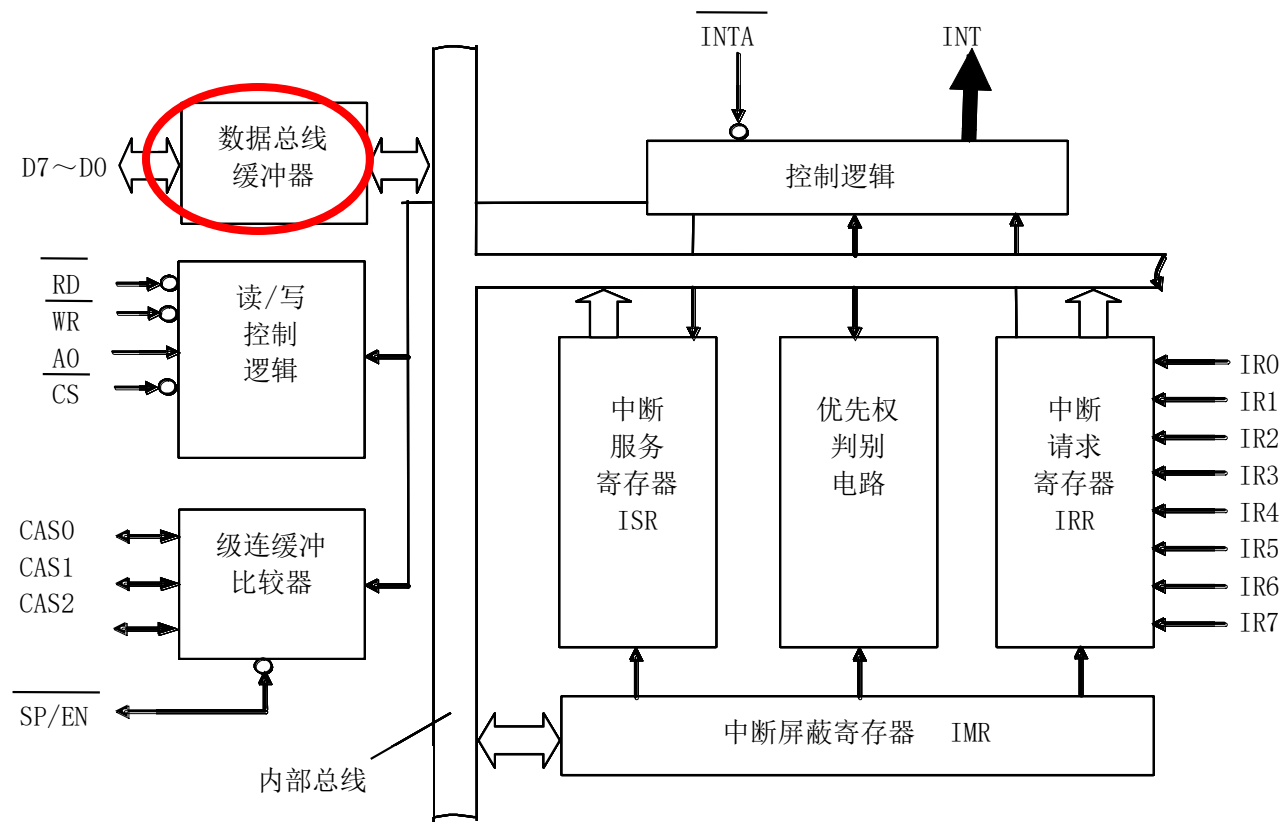
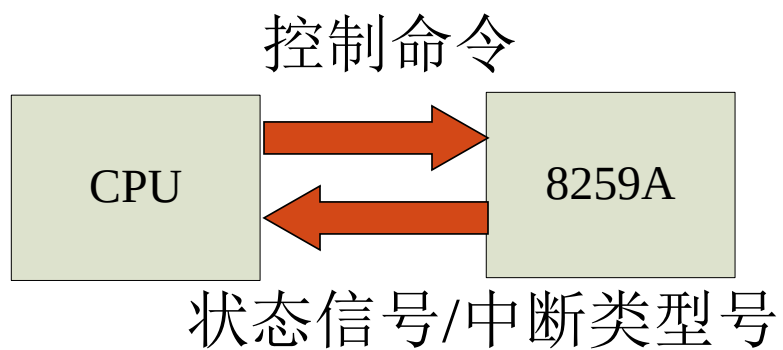
- 优先权判断器
 - 根据控制逻辑规定的优先权级别和IMR的内容
 - 判断出IRR中提出中断的优先权最高的中断请求位，CPU响应后送ISR



	IRR7			IRR4			IRR0	
IRR	0	0	0	1	0	0	0	1
ISR	0	0	0	0	0	0	0	1

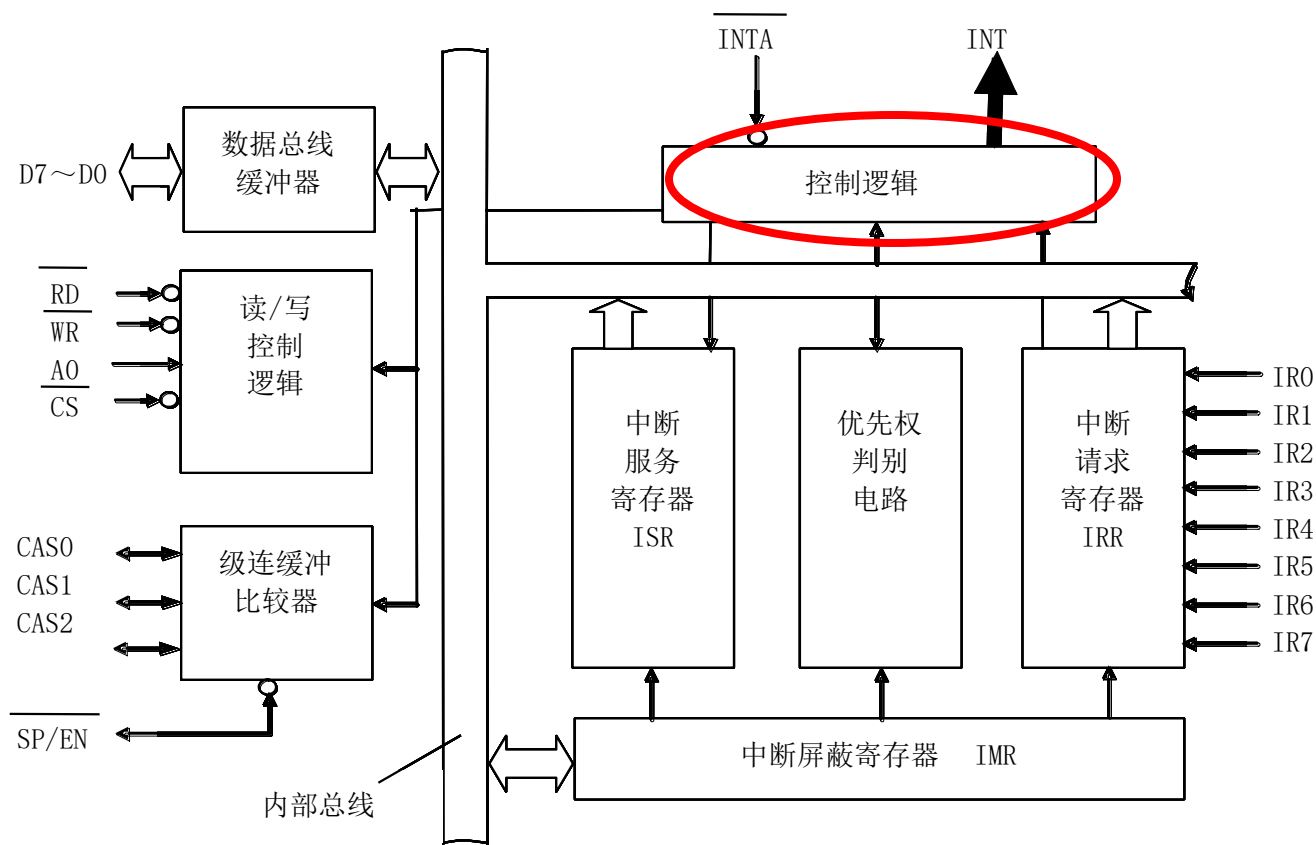
内部结构

- 数据总线缓冲器
 - 双向缓冲
 - 连接管脚
 - 8位芯片



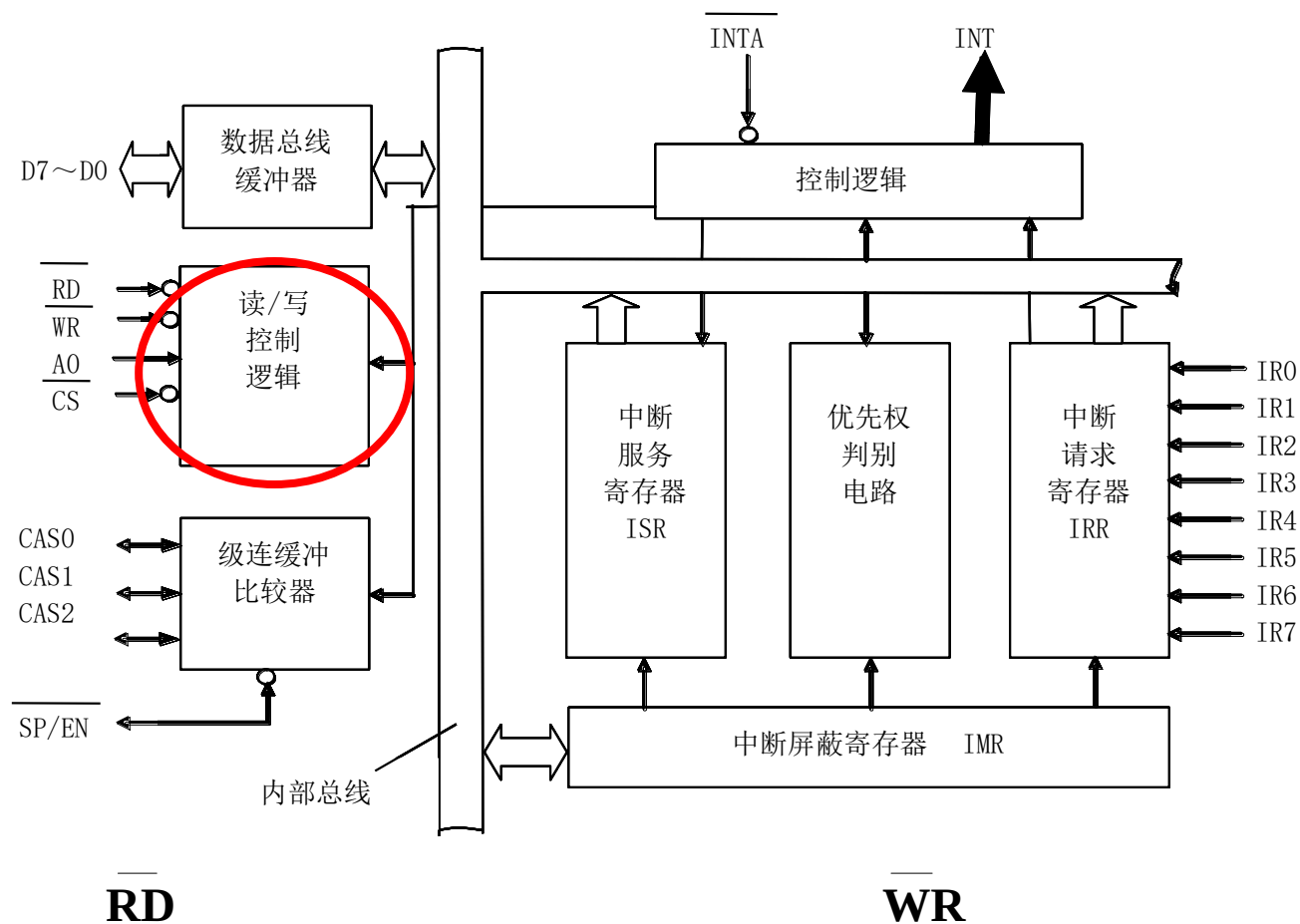
内部结构

- 控制逻辑
 - 初始化命令字
 - ICW1, ICW2, ICW3, ICW4
 - 操作命令字
 - OCW1, OCW2, OCW3
- 工作方式管理
- 连接管脚



内部结构

- R/W 逻辑
 - 连接管脚
 - 2个端口地址
 - 通过ICW2 设置中断类型号(比如:
30H-37H)

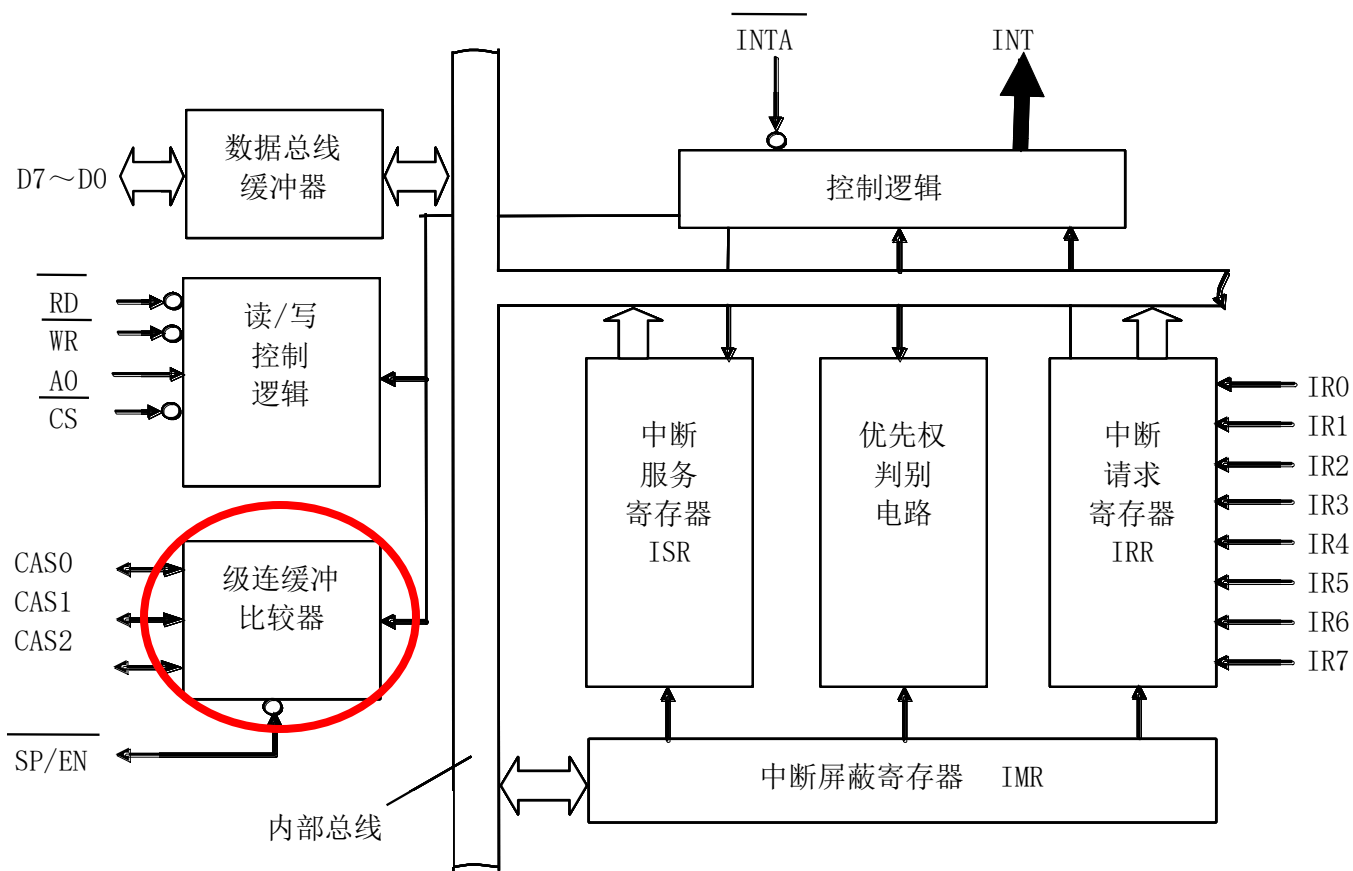


RD		WR	
A0	0	IRR ISR 最高级(中断状态字)	ICW1 OCW2 OCW3
	1	OCW1(IMR)	ICW2,3,4 OCW1

中断状态字ISW: OCW3, 读当前有无中断, 以及查询CPU在处理哪一位的中断处理程序

内部结构

- 级联缓冲器
 - 连接管脚
 - 主片输出识别码
 - 从片输入识别码
- 通过ICW3进行级联控制设置（主片给出从片连接的对应管脚，从片给出识别地址000~111）



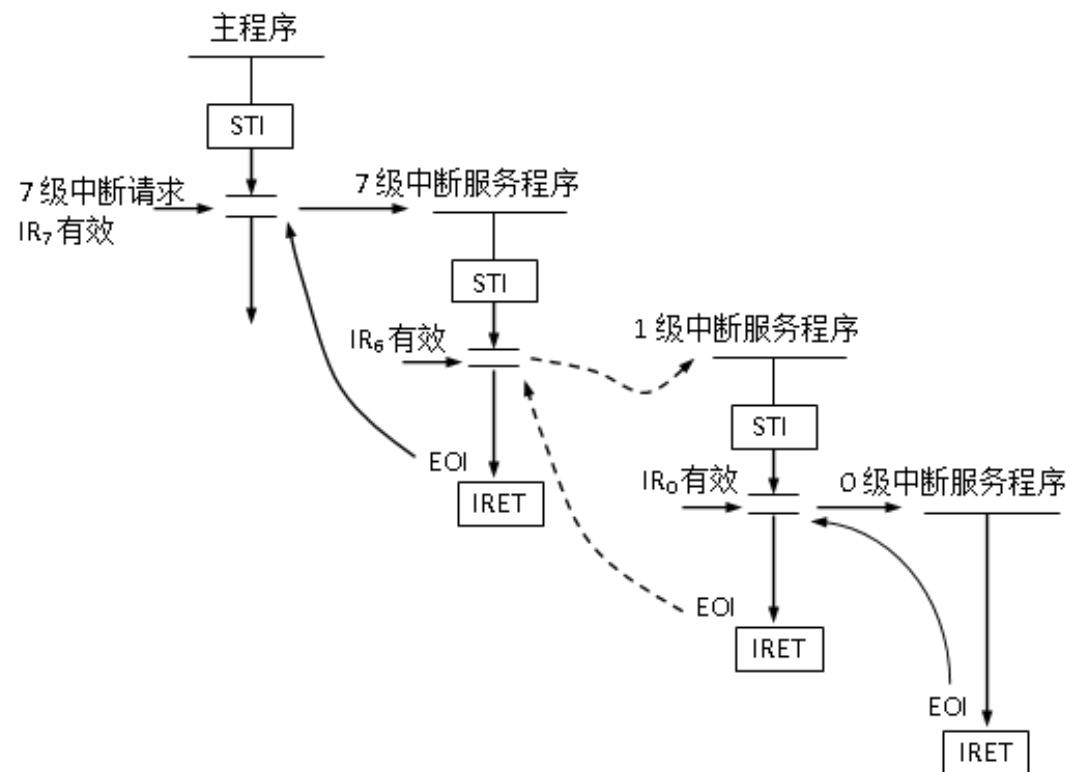
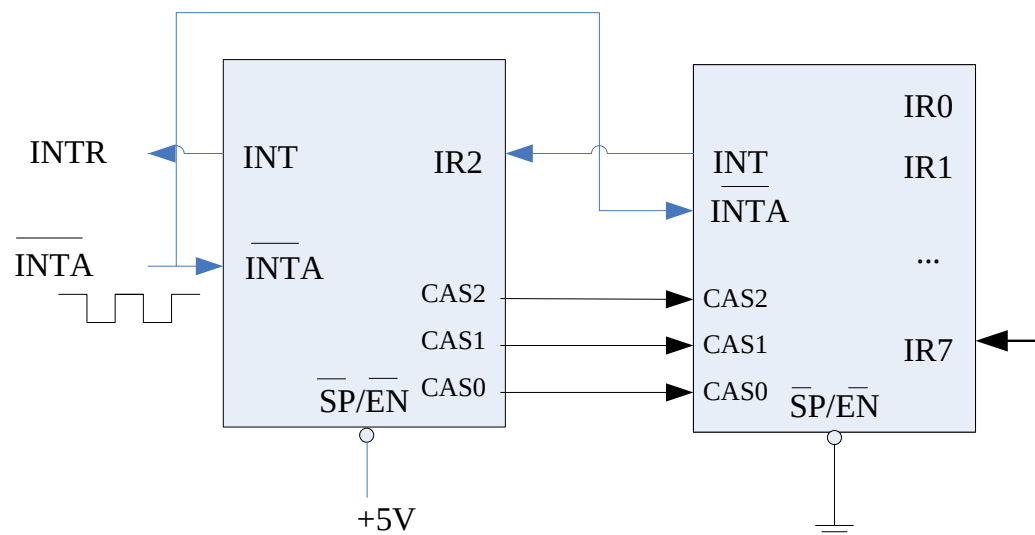
中断管理

- 优先级管理
- EOI: 中断结束，清除ISR相应位

优先级管理

- 完全嵌套方式

- 固定优先级，IR0 最高, IR7 最低
- 中断嵌套
- 中断源的中断服务程序完成之前，与它同级或优先权更低的中断源的申请就被屏蔽
- 只有优先权比它高的中断源的申请才是允许的
- 如果对应的ISR_i=1，则IR_i 不能被处理



优先级管理

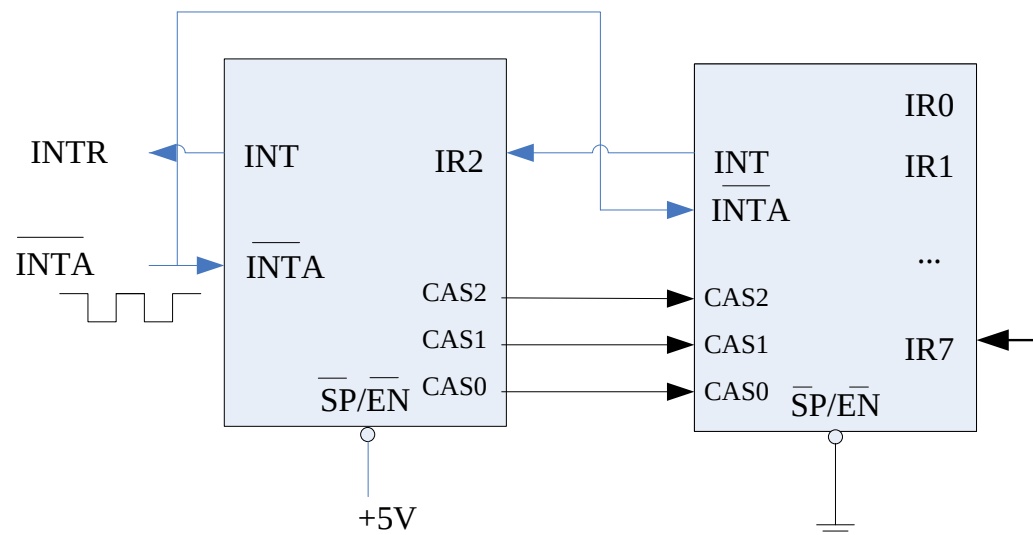
- 特殊完全嵌套方式

- 用于级联的主片
- 即使 $ISR_i=1$ ，依然处理 IR_i 的请求
- 优先级顺序
 - 主从片设置为完全嵌套方式

主 $IR_0 > 主IR_1 > 主IR_2$ (从 $IR_0 = 从IR_1 = \dots = 从IR_7$) $> 主IR_3 > \dots > 主IR_7$

- 主片设置为特殊完全嵌套方式，
从片设置为完全嵌套方式

主 $IR_0 > 主IR_1 > 主IR_2$ (从 $IR_0 > 从IR_1 > \dots > 从IR_7$) $> 主IR_3 > \dots > 主IR_7$



优先级管理

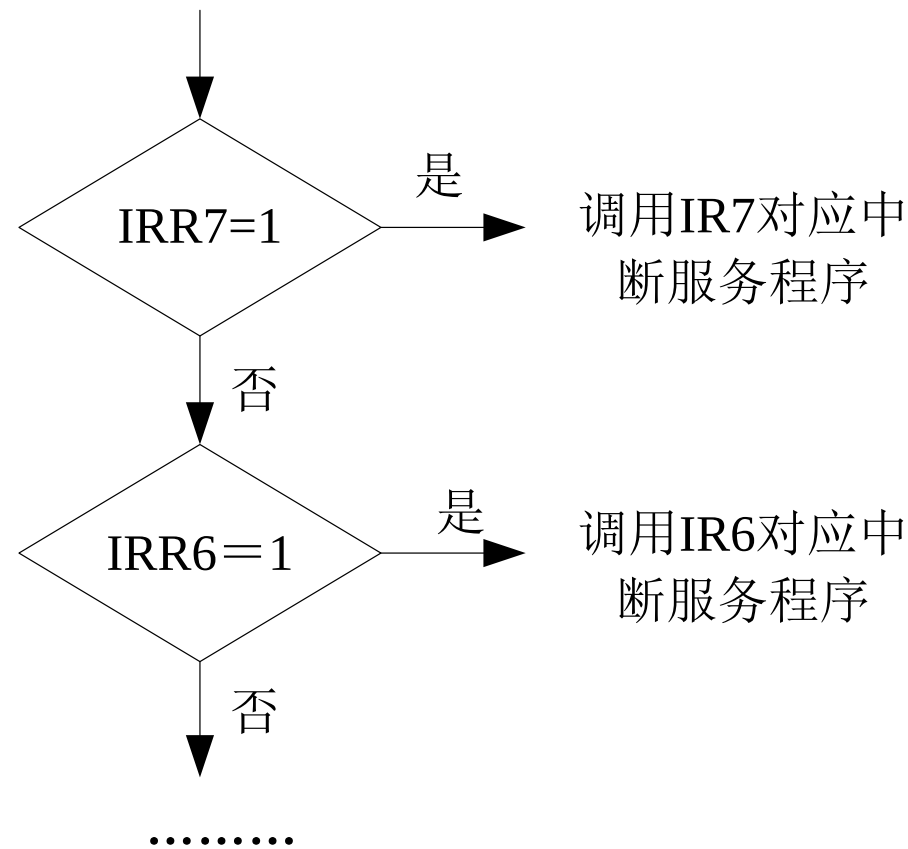
- 优先级循环
 - 自动循环
 - 按照IR0、IR1、IR2、...、IR7的顺序，优先级从高到低，但IRi中断服务执行后，优先级自动降为最低，原来比它低一级的中断则为最高级，依次排列
 - E.g. 当IR3被服务后，IR4变成最高优先级
 - IR4>IR5>IR6>IR7>IR0>IR1>IR2>IR3
 - 特殊循环
 - 初始优先级可设置

优先级管理

- 中断屏蔽方式
 - 普通屏蔽方式
 - $IMR_i = 1$, IR_i 请求被屏蔽

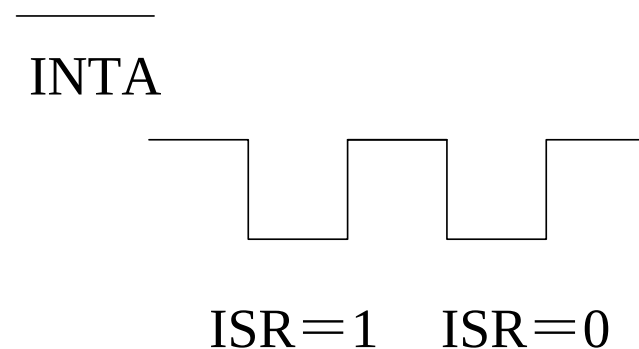
优先级管理

- 查询方式
 - 不使用INT输出，IMR=FFH
 - 读取IRR状态
 - 软件控制

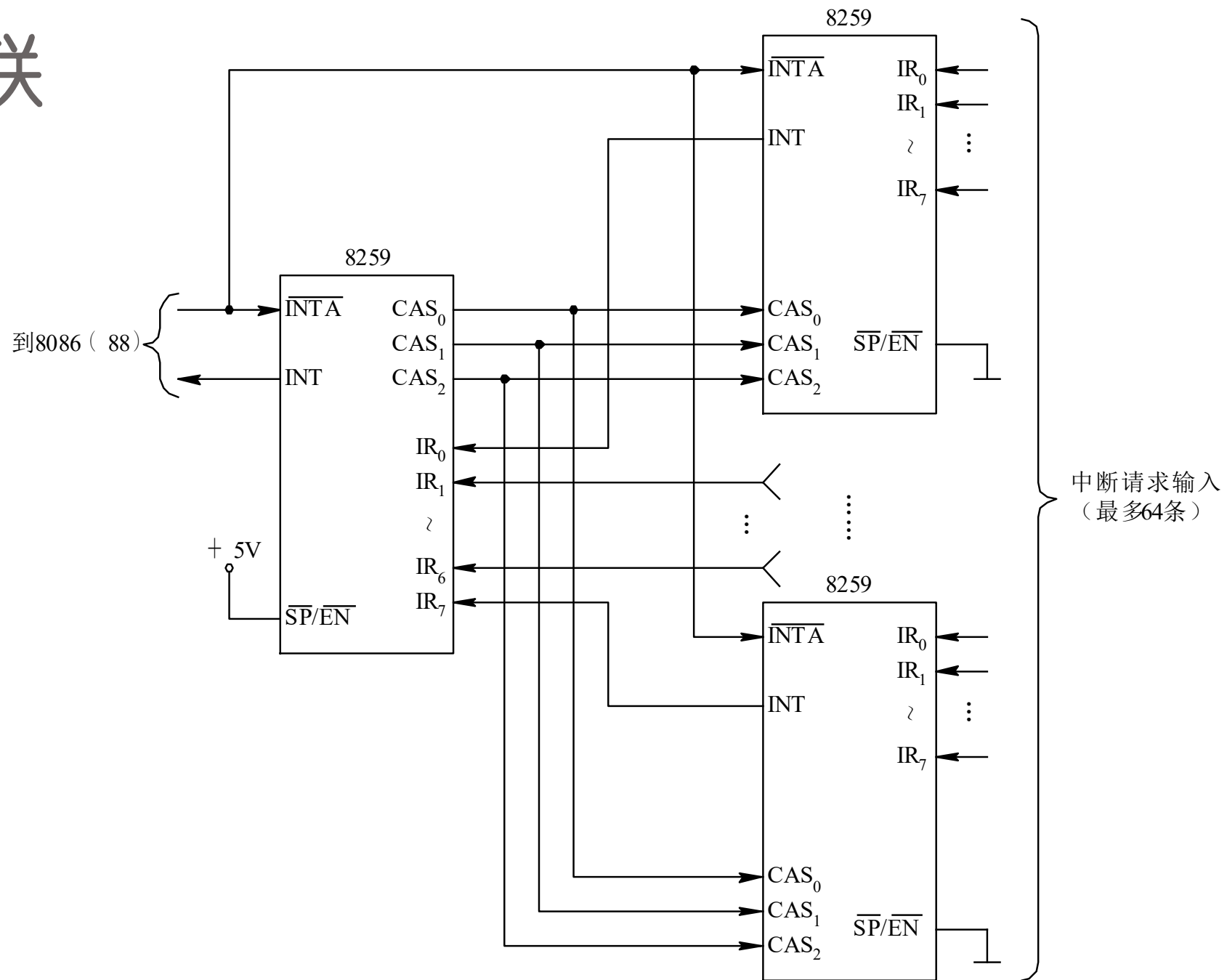


中断结束

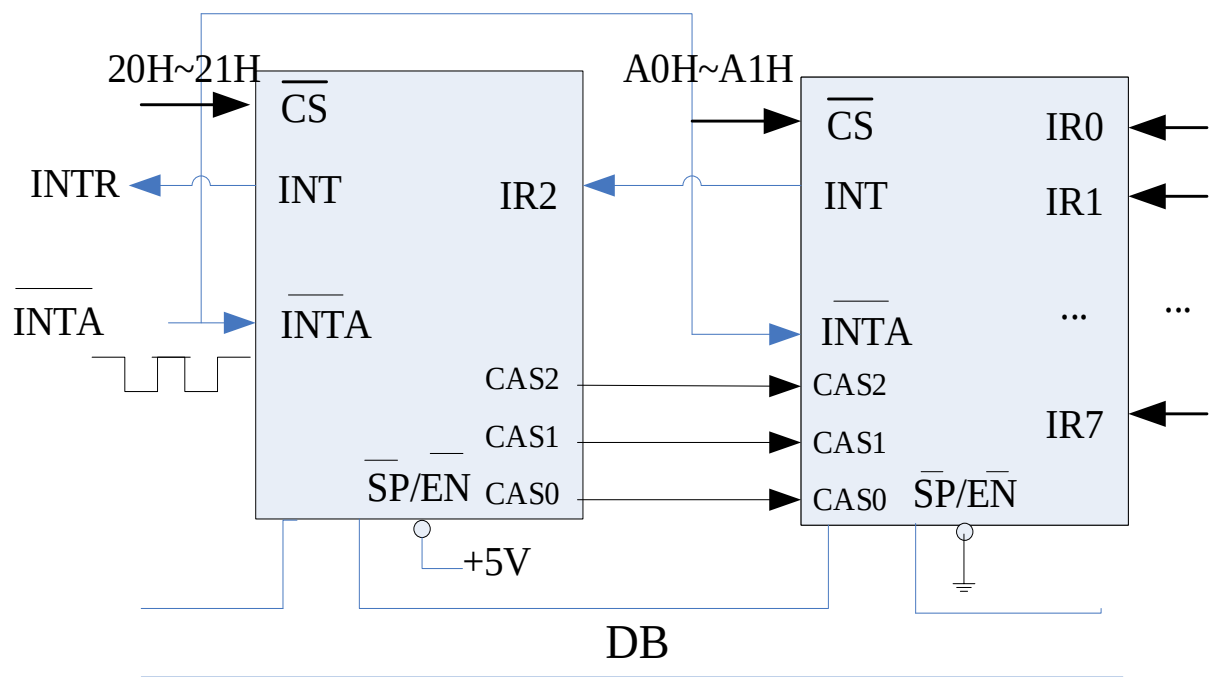
- 中断结束方式（EOI）：ISR复位
 - AEOI 自动EOI
 - 在 $\overline{\text{INTA}}$ 第二个低电平自动清除对应的ISR位
 - 通常用于只有一片8259A
 - 不要求中断嵌套的情况下
 - 其他EOI (中断返回前)
 - 通过编程发复位命令



中断级联



中断级联应答过程

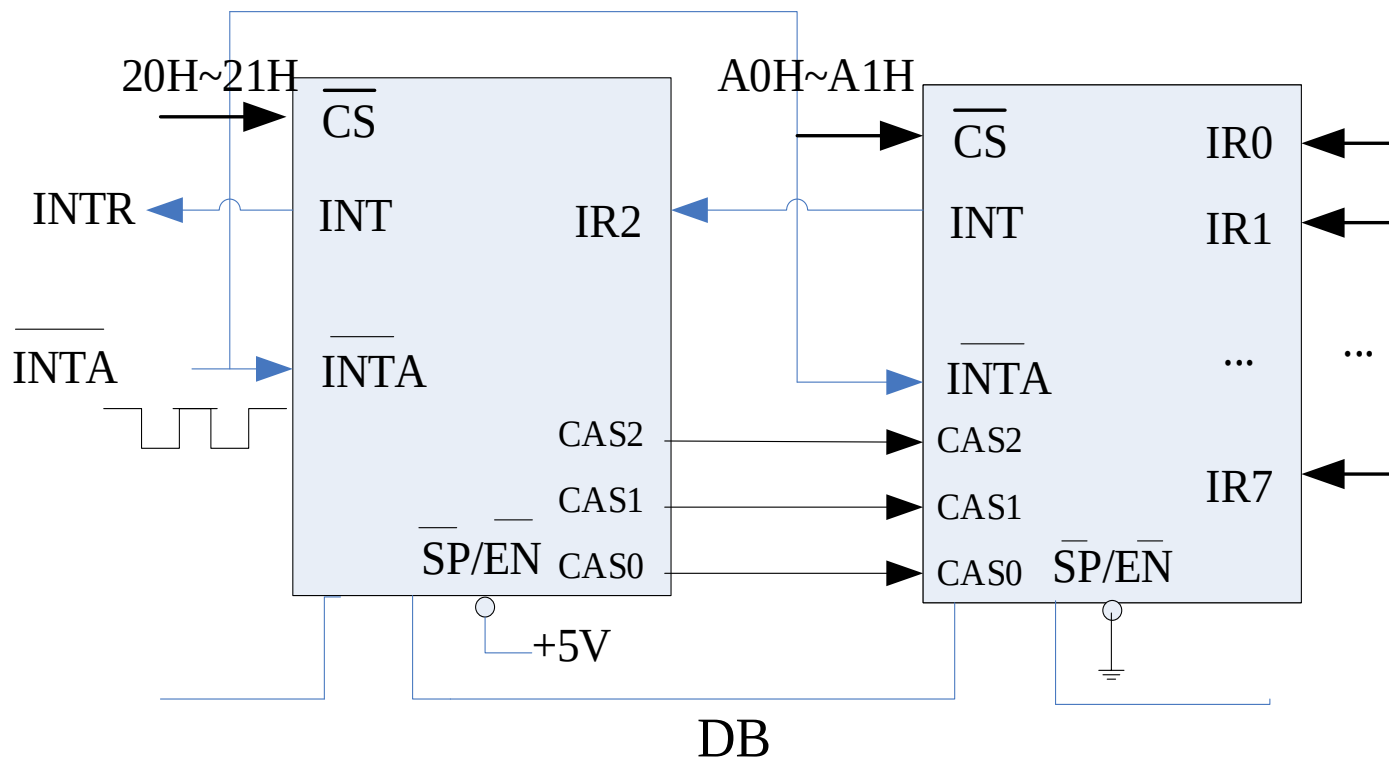


- 从片发**INTR** 给主片
- 主片根据优先级判定发**INTR** 给CPU
- CPU 发第一个**INTA** 低电平信号, 主片ISR和从片ISR对应位置位, 对应IRR复位
- CPU 发第二个**INTA** 低电平, 选中的从片输出中断类型号到数据总线
- 执行中断服务子程序
- 发**2** 个**EOI** 命令给主片和从片, 分别清除ISR对应位

使用可屏蔽中断

使用可屏蔽中断

- 硬件
 - 正确连接CPU, 8259A 和外部设备
- 软件
 - 设置中断向量
 - 编写中断服务子程序



设置中断向量

- 中断向量表
 - $N*4, N*4+1$ 偏移地址
 - $N*4+2, N*4+3$ 段地址
- 设置方式
 - DOS 子程序
 - 其他

	...
4N	两字节
4N+1	偏移地址
4N+2	两字节
4N+3	段地址
	...

DOS子程序

- 设置中断向量: INT 21H, 25H功能

AL = 中断号

DS:DX = 中断向量

AH = 25H

INT 21H

- 获取中断向量: INT 21H, 35H功能

AL = 中断号

AH = 35H

INT 21H

ES:BX = 中断向量

- **Examples**

- 中断类型号: **N**

- 中断服务子程序: **INTRAD**

```
MOV AX, SEG INTRAD
```

```
MOV DS, AX
```

```
MOV DX, OFFSET INTRAD
```

```
MOV AL, N
```

```
MOV AH, 25H
```

```
INT 21H
```

中断服务子程序

- 1) 程序开始必须保护中断现场：一系列**PUSH**指令将CPU各个寄存器的值压入栈中
- 2) 若允许中断嵌套，则用**STI**指令来开中断，使得 $IF=1$
- 3) 执行中断处理程序
- 4) 用**CLI**关中断，使得 $IF=0$ ，禁止响应其它中断
- 5) 给中断命令寄存器（8259A）送中断结束命令**EOI**
- 6) 恢复中断时的现场：用**POP**指令恢复CPU各个寄存器的值
- 7) 用中断返回指令**IRET**返回主程序
 - 自动弹出断点地址到IP, CS
 - 弹出FR

与子程序调用过程的区别？堆栈的变化？

中断例子

- 每1秒显示“HELLO”
- 如何实现：利用中断类型1CH进行处理，系统每秒调用18.2次，则调用18.2次需要1秒钟
- 原来的中断程序只有**IRET**指令
 - 用自己的程序替换原来的中断程序
 - 中断服务子程序：计数18次后输出HELLO
- **堆栈状态**

主程序

获取之前的中断向量



入栈保存



替换新的中断向量



延时



原始中断向量出栈



恢复原始中断向量

中断例子

```
DATA SEGMENT
    COUNT DW 1
    MESS DB 'HELLO!',0DH,0AH,'$'
DATA ENDS

STACK SEGMENT
    ST1 DB 100 DUP(?)
STACK ENDS
```

主程序

获取之前的中断向量



入栈保存



替换新的中断向量



延时



原始中断向量出栈



恢复原始中断向量

中断例子

CODE SEGMENT

MAIN PROC **FAR**

ASSUME CS:CODE, DS:DATA, ES:DATA,
SS:STACK

START:

MOV AX, STACK

MOV SS, AX

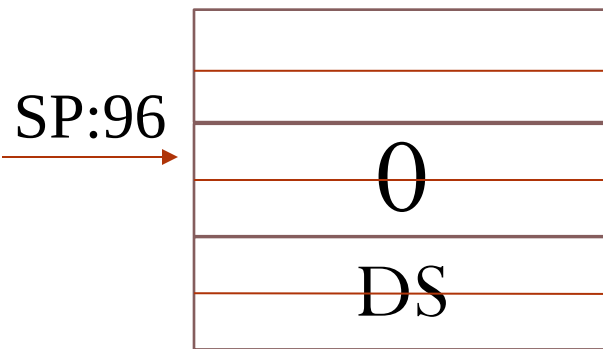
MOV SP, (OFFSET ST1+(LENGTH ST1));100

PUSH DS

SUB AX, AX

PUSH AX

SP:96



程序段前
缀段地址

主程序

获取之前的中断向量



入栈保存



替换新的中断向量



延时



原始中断向量出栈



恢复原始中断向量

中断例子

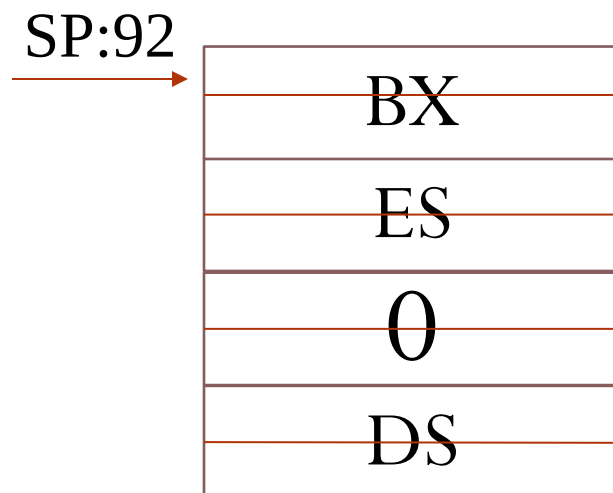
MOV AL, 1CH; 获取原中断向量

MOV AH, 35H

INT 21H; 放入ES:BX

PUSH ES;保存原中断向量

PUSH BX



程序段前
缀段地址

主程序



中断例子

MOV DX, OFFSET RING

MOV AX, SEG RING

MOV DS, AX

MOV AL, 1CH;设置中断向量

MOV AH, 25H

INT 21H

主程序

获取之前的中断向量

入栈保存

替换新的中断向量

延时

原始中断向量出栈

恢复原始中断向量

中断例子

```
STI
MOV DI, 0FFH ;延迟
DELAY: MOV SI, 0FFFFH
DELAY1: DEC SI
      JNZ DELAY1
      DEC DI
      JNZ DELAY
```

主程序



中断例子

CLI

POP DX; 取原中断向量偏移地址

POP DS; 取原中断向量段地址

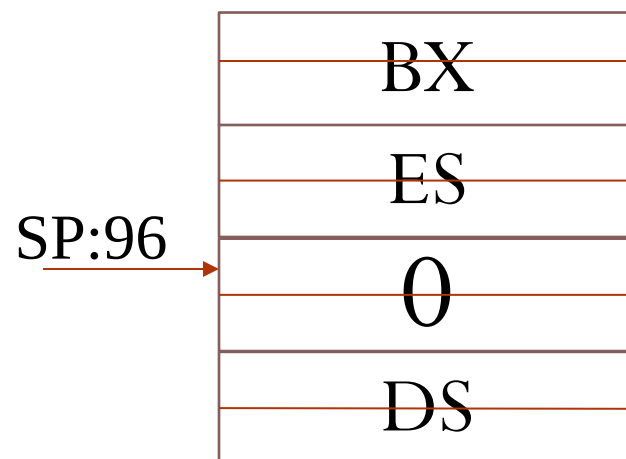
MOV AL, 1CH

MOV AH, 25H

INT 21H

RET

MAIN ENDP



程序段前
缀段地址

主程序

获取之前的中断向量

入栈保存

替换新的中断向量

延时

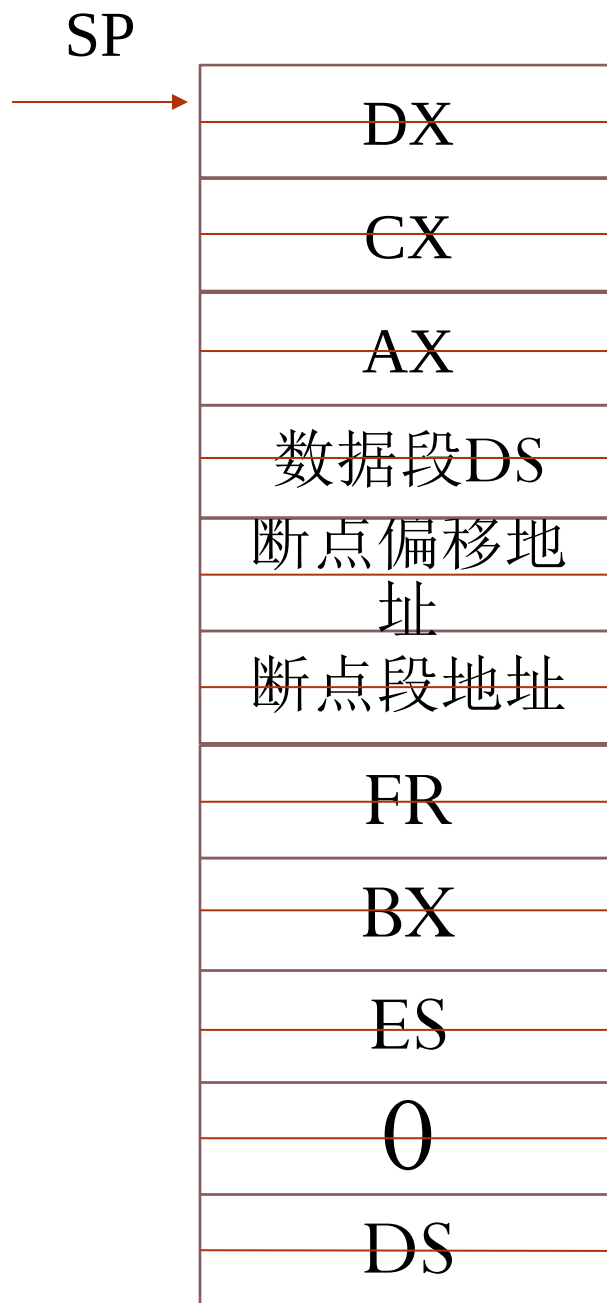
原始中断向量出栈

恢复原始中断向量

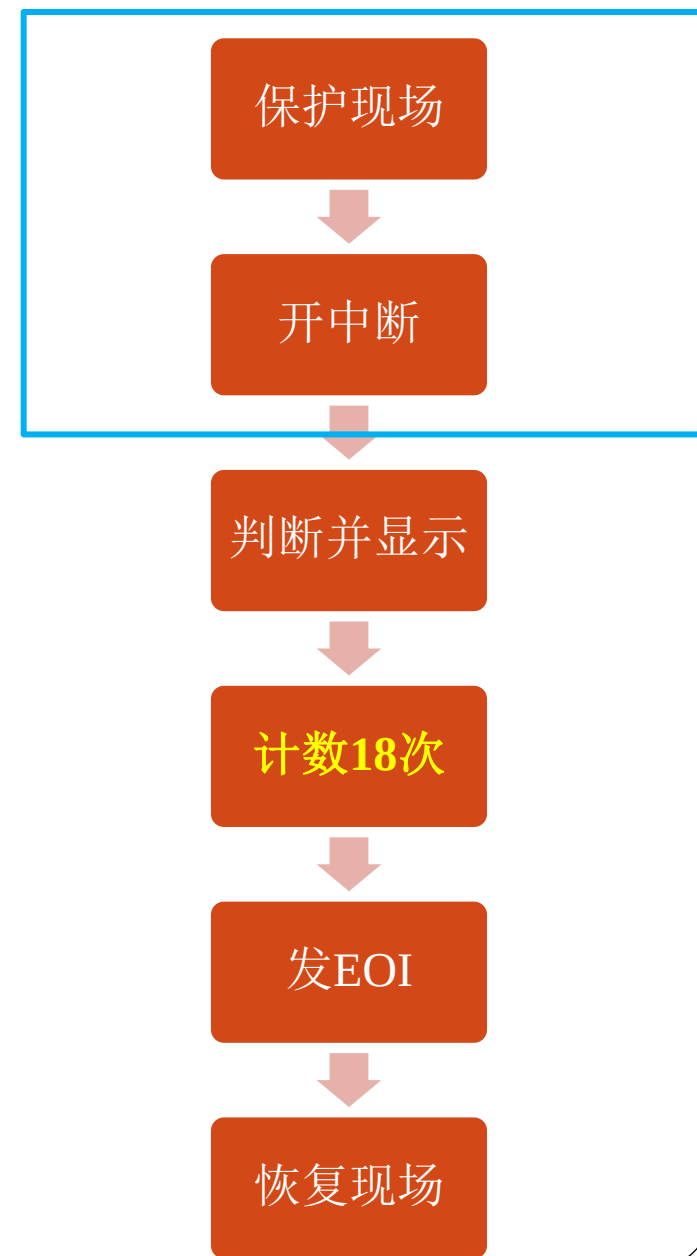
中断例子

```
RING PROC NEAR
PUSH DS
PUSH AX
PUSH CX
PUSH DX;

MOV AX, DATA
MOV DS, AX
STI
```



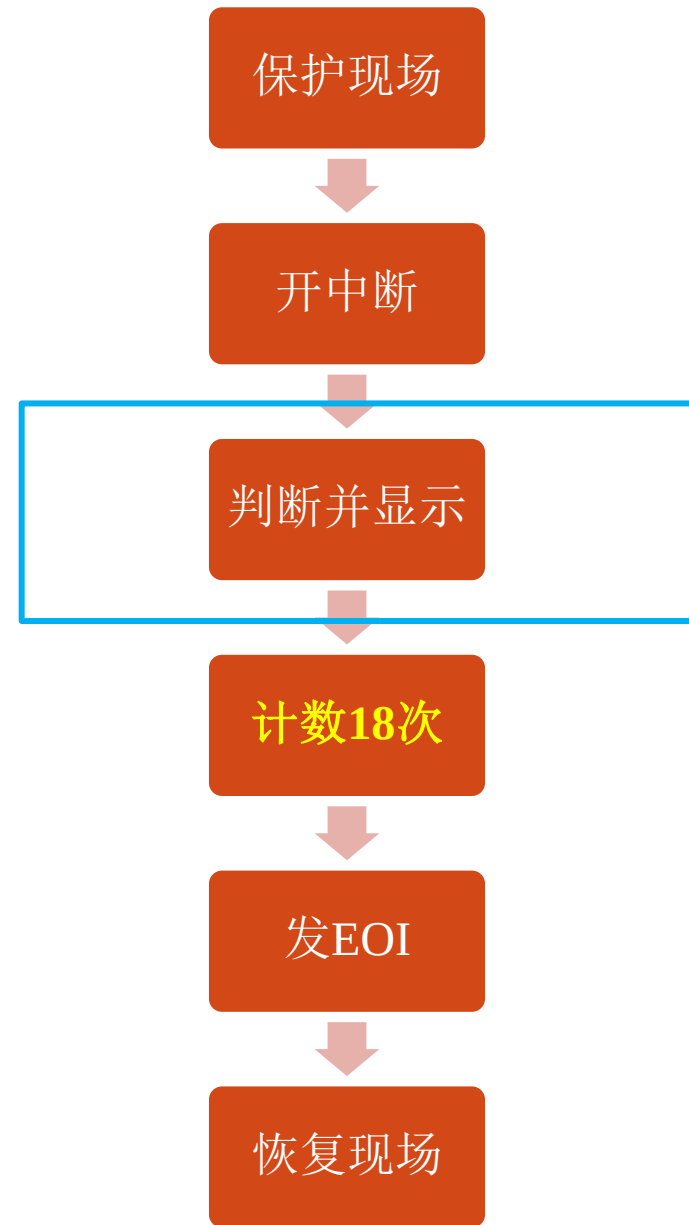
中断服务子程序



中断例子

```
DEC COUNT;1秒计数  
JNZ EXIT; COUNT为0则显示  
MOV DX, OFFSET MESS  
MOV AH, 09H;显示信息  
INT 21H
```

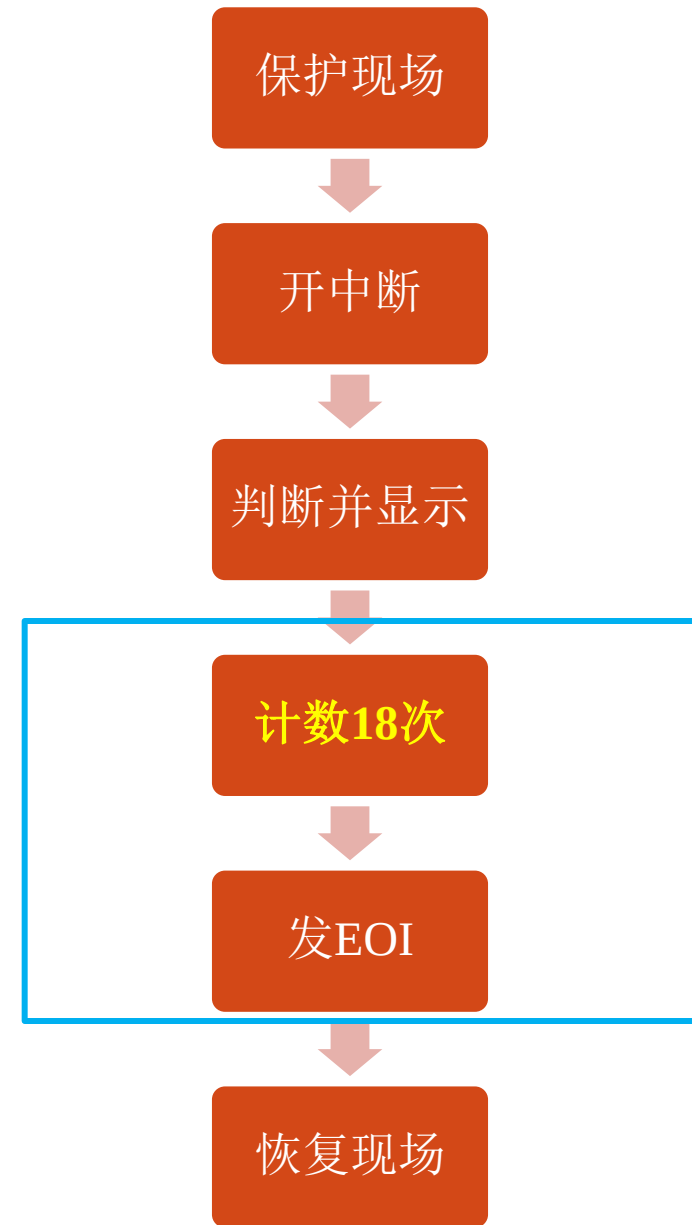
中断服务子程序



中断例子

```
MOV COUNT, 18  
EXIT:  
CLI  
  
MOV AL, 20H  
OUT 20H, AL; 发EOI
```

中断服务子程序



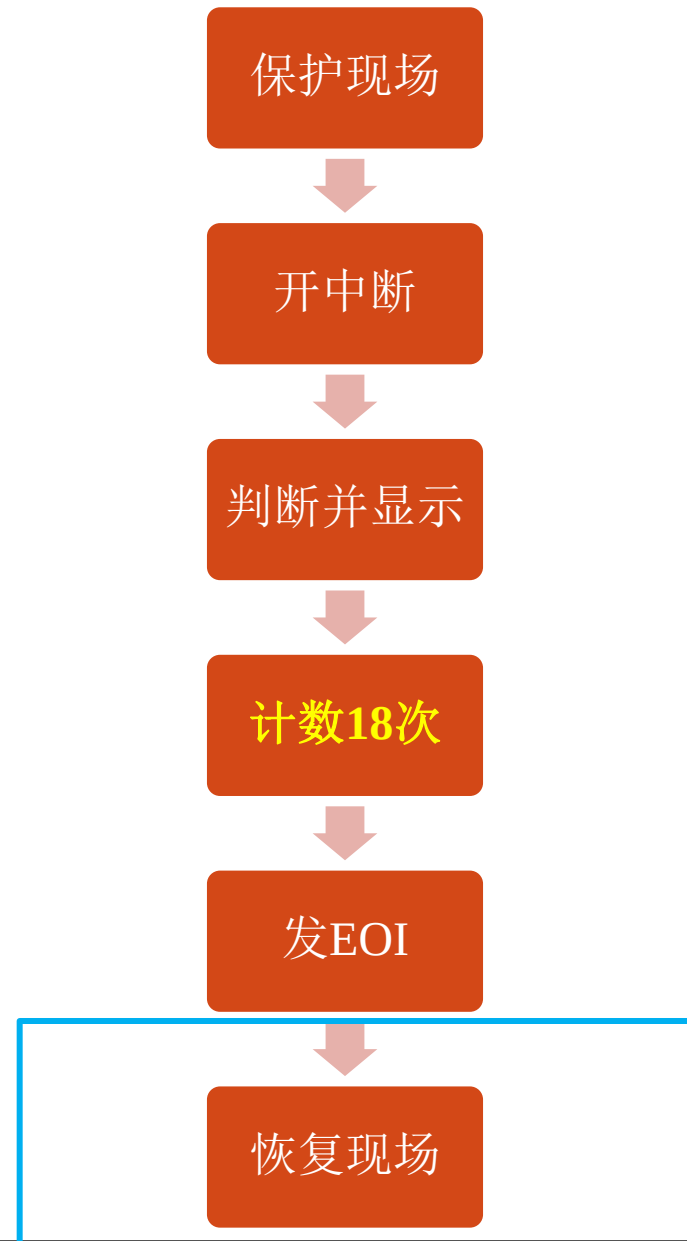
中断例子

```
POP DX
POP CX
POP AX
POP DS
IRET
RING ENDP

CODE ENDS
END START
```



中断服务子程序



练习

- 8259A中的中断服务寄存器某一位ISR_i如果置1，则8259A不向CPU转发相应引脚IR_i的中断请求？
- 完全嵌套方式指的是8259A引脚所接中断源的优先级固定，这时引脚编号越大，优先级越高？
- 8086系列微机的中断优先级是根据中断类型码的大小来排列的，类型码越小，优先级越高，类型码越大，优先级越低？
- CPU在执行程序时发生了中断，则被打断处的下一条指令就称为断点？
- 微机中使用了主从2片8259A，故可有16个可屏蔽中断源？

